Industrial Training – Project Report

(MCA-C651)

on

Autism Spectrum Disorder Prediction using Machine Learning and Explainable AI

Submitted in partial fulfillment of the requirements

for the award of the degree of

Master of Computer Applications (MCA)

Uttaranchal School of Computing Sciences

UTTARANCHAL UNIVERSITY DEHRADUN | NAAC GRADE A+

BATCH 2024-2026

Submitted to:

Dr. Amar Jeet Rawat

Associate Professor

Uttaranchal School of Computing Sciences

Submitted by

Sanya Singh

UU2420000114

MCA -IV (B)

ACKNOWLEDGEMENT

The most awaited moment of any endeavor is its successful completion, but nothing can be done successfully if done alone. Success is the outcome of continuous technical mentoring, guidance, and support from various individuals, and I express my sincere gratitude to all those who contributed to the successful completion of this project.

First and foremost, I would like to thank **Prof. (Dr.) Sonal Sharma, Director, USCS,** for providing a healthy and encouraging environment for study.

I am extremely thankful to **Prof.(Dr.) Sameer Dev Sharma, Head of Department, USCS,** and my project mentor **Dr. Amar Jeet Rawat,** for providing me with this opportunity and for their valuable technical mentorship, continuous guidance, and expert insights throughout the project development process. Their support at every stage significantly enhanced my understanding and helped me successfully complete this work.

I also express my sincere gratitude to my colleagues for their valuable discussions and support, which helped me in giving a final shape to this project.

Sanya Singh
UU2420000114
MCA IV B

STUDENT DECLARATION

I hereby declare that the project report entitled **Autism Spectrum Disorder Prediction using Machine Learning with Explainable AI** submitted by me. The project was done under the guidance of **Dr. Amar Jeet Rawat** in partial fulfillment of the requirements for the award of the degree of **Master of Computer Applications** from **Uttaranchal University.**

I further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this university or any other university or institute.

Dated: 19th April, 2026

Sanya Singh

UU2420000114

MCA IV B

Uttaranchal School of Computing Sciences

CERTIFICATE

This is to certify that the project work entitled **"Autism Spectrum Disorder Prediction using Machine Learning and Explainable AI"** is a bonafide work carried out by **Sanya Singh (UU2420000114)** submitted in partial fulfilment for the award of the degree in **MASTER OF COMPUTER APPLICATIONS** of **Uttaranchal University, Dehradun** during the academic year 2025-26.

It is verified that the project files and project report (PDF) are uploaded on GitHub git@github.com:sanya1110/Autism-Spectrum-Disorder-Prediction-using-Machine-Learning-and-Explainable-AI-SHAP-.git

Dr. Amar Jeet Rawat

Associate Professor

USCS

INDUSTRIAL TRAINING CERTIFICATE

Eunora Technologies Private Limited
TIIR Building, NIT Rourkela, Odisha - 769008
Ph- 7077093396 | Mail: umesh.csahoo@eunoratechnologies.in
Co-founder and COO : Umesh Chandra Sahoo

INTERNSHIP COMPLETION

This is to certify that **Ms. SANYA SINGH** was working as a **Machine learning Intern** with Eunora Technologies Private Limited from **06th Feb 2026** to **07th April 2026**

We wish her all the best in her future endeavours.

Warm regards,

From Eunora Technologies Private Limited

Signed by
Umesh Chandra Sahoo
Chief Operational Officer,
Eunora Technologies Private Limited.
Dated: 09th April 2026

PREFACE

This project report titled "**Autism Spectrum Disorder Prediction using Machine Learning with Explainable AI"** has been prepared as a partial fulfilment for the award of the degree of **Master of Computer Applications**. The objective of this project is to apply theoretical concepts to real-world problems and to gain practical exposure in design, development, and implementation

This project focuses on the development of an intelligent and interpretable system for the early prediction of Autism Spectrum Disorder (ASD) using Machine Learning and Explainable Artificial Intelligence. The problem addressed in this work is the limitation of traditional autism diagnosis methods, which are by their nature require expert professionals, and are not easily accessible in all regions. Early detection of ASD is crucial for timely intervention; however, delays in diagnosis can negatively impact developmental outcomes.

The objective of this project is to design a data-driven screening tool that can accurately predict the likelihood of ASD based on behavioral, demographic, and screening data. The system also aims to provide explainable predictions using SHAP (SHapley Additive exPlanations), ensuring transparency and improving trust among users such as healthcare professionals and caregivers.

The key outcome of this project is the development of a reliable and efficient predictive system capable of supporting early screening of ASD. The project files are uploaded on GitHub git@github.com:sanya1110/Autism-Spectrum-Disorder-Prediction-using-Machine-Learning-and-Explainable-AI-SHAP-.git

Dr. Amar Jeet Rawat
Associate Professor

Sanya Singh
UU2420000114

TABLE OF CONTENTS

S. No.	Heading	Page No.
	Acknowledgement	I
	Declaration	II
	Certificate of Originality	III
	Industrial Training Certificate	IV
	PREFACE	V
	Table of Contents	VI
	List of Figures	VIII
	List of Tables	IX
1.	**INTRODUCTION**	10
2.	**OBJECTIVES**	12
3.	**SYSTEM ANALYSIS**	13
	3.1 Identification of Need	16
	3.2 Preliminary Investigation	18
	3.2.1 Existing System	20
	3.2.2 Proposed System	22
	3.3 Feasibility Study	25
	3.3.1 Technical Feasibility	27
	3.3.2 Economical Feasibility	29
	3.3.3 Operational Feasibility	31
	3.4 Project Planning	33
	3.5 Project Scheduling	37
	3.5.1 Gantt Chart	40
	3.5.2 PERT Chart	41
	3.6 Software Requirement Specification	43
	3.6.1 Functional Requirements	44
	3.6.2 Non-Functional Requirements	44
	3.7 Software Engineering Paradigm	45
	DATA MODELS	50
	3.8.1 Flow Chart	50
	3.8.2 Sequence Diagram	53
	3.8.3 Activity Diagram	56
	3.8.4 ER Diagram	59
	3.8.5 Class Diagram	62
	3.8.6 DFD (Level 0, 1, 2)	65
4.	**SYSTEM DESIGN**	68
	4.1 Modules Description	68
	Module 1: Data Collection Module	68
	Module 2: Data Preprocessing Module	69
	Module 3: Exploratory Data Analysis (EDA) Module	71
	Module 4: Model Training and Evaluation Module	88
	Module 5: Prediction Module	94
	4.2 User Interface Design	99
5.	**TESTING**	100
	5.1 Testing Techniques and Testing Objectives	101

	5.2 Testing Reports	103
6.	**SYSTEM SECURITY ANALYSIS**	107
7.	**COST ESTIMIATION**	112
8.	**FUTURE SCOPE**	114
9.	**APPENDICES**	116
	9.1 Coding	
	9.2 Bibliography	
	9.3 Plagiarism Report	

LIST OF FIGURES

S.No	Number	Name
1	Fig 3.5.1.1	Gantt Chart
2	Fig 3.5.2	Pert Chart
3	Fig 3.8.1	Flow Chart
4	Fig 3.8.2	Sequence Diagram
5	Fig 3.8.3	Activity Diagram
6	Fig 3.8.4	ER Diagram
7	Fig 3.8.5	Class Diagram
8	Fig 3.8.6.1	0 Level DFD
9	Fig 3.8.6.2	1 Level DFD
10	Fig 3.8.6.3	2 Level DFD
11	Fig 4.4.1.1	Attributes for the dataset used
12	Fig 4.4.1.2	Figure shows the label encoder
13	Fig 4.4.1.3	SMOTE balances the classes
14	Fig 4.4.1.4	Age Distribution of Individuals
15	Fig 4.4.1.5	Gender Distribution
16	Fig 4.4.1.6	Class Distribution (ASD vs Non-ASD)
17	Fig 4.4.1.7	Distribution of Age with mean & median
18	Fig 4.4.1.8	Distribution of screening result score with mean median
19	Fig 4.4.1.9	Box Plot for Age Showing Distribution and Outliers
20	Fig 4.4.1.10	Box Plot for Result Showing Distribution and Outliers
21	Fig 4.4.1.11 – Fig 4.4.1.27	Count Plots
22	Fig 4.4.1.28	Correlation Heatmap Showing Relationships Between Features
23	Fig 4.4.1.29	Print(x) show the train attributes
24	Fig 4.4.1.30	Print(y) show the train attributes
25	Fig 4.4.1.31	RandomizedSearchCV Applied on XGBoost Model
26	Fig 4.4.1.32	Selection of Best Performing Model Based on Cross-Validation
27	Fig 5.2.1.	Evaluation Metrics of the Final Model on Test Data
28	Fig 5.2.2	SHAP Summary Plot Showing Feature Impact on Prediction
29	Fig 5.2.3	SHAP Feature Importance Bar Plot
30	Fig 5.2.4	SHAP Force Plot Explaining Individual Prediction

LIST OF TABLES

S.No.	Number	Name
1	Table 3.3.1	Software dependency used in the project
2	Table 3.5.1	Gantt based schedule of learning and project activities
3	Table 7.1	Cost Estimation

1. INTRODUCTION

Autism Spectrum Disorder (ASD) is a neurodevelopmental condition that affects communication, behavior and social interaction. This condition is seen in different degrees and early identification of it in an individual is very crucial for effective intervention. If diagnosis is delayed, it can affect social development, emotional well being of the individual and causes negative effect on the overall quality of life.

During the initial phase of the training, multiple modules were covered such as Python fundamentals, data structures, object-oriented programming (OOP), data analytics, statistical foundations, visualization techniques and core Machine Learning concepts such as supervised learning, model evaluation, cross-validation, and ensemble methods. Libraries such as NumPy, Pandas, Matplotlib, Seaborn, and Scikit-learn were used to understand the practical use of them. The module-based learning approach helped me to develop strong conceptual base in order for me to apply it to the implementation phase.

In the second phase of the program, interns were made to select a problem statement and then develop a predictive or analytical solution to that problem with whatever they have learned through the learning phase. I proposed my project as **"Autism Spectrum Disorder Prediction using Machine Learning and Explainable AI (SHAP)".**

The proposed project aims to develop a machine learning-based predictive system which analyses the behavioral and demographic data to estimate the probability of Autism. Classification algorithms and data preprocessing techniques are used in the project which shows how technology supports mental health awareness and early detection efforts.

The selection of this topic by me is motivated by the increasing concerns on mental health globally. The objective is to not just gaining technical proficiency in Machine Learning but, how can those techniques have a better

impact on the society.

Machine learning based classification models analyze patterns in behavioral and demographic data to predict the likelihood of Autism. The usage of algorithms such as Logistic Regression, Decision Trees, Random Forest, aur Support Vector Machines (SVM) helped to develop a predictive system which can support the healthcare professionals.

However, the major challenge of machine learning in the domain of healthcare is the lack of interpretability. A lot of the models work like “black box” where predictions are gained but the reasoning behind the is never explained by the model. A domain so critical cannot compromise with transparency as it can affect the trust in such systems and restrict its practical adoption. For this problem, Explainable Artificial Intelligence (XAI) brings solutions.

2. OBJECTIVES

The objectives of this industrial training are:

1. Develop an Accurate Prediction Model

To build and train machine learning models so that they predict the likelihood of autism spectrum disorder in individuals based on behavioral, demographic and screening data.

2. Improve Early Detection and Screening

To assist in the early identification of autism by providing a fast and data-driven screening tool which supports clinicians and caregivers.

3. Enhance Model Transparency using SHAP

To apply explainability to the model so that it provides explanations which shows how each feature contributes to the outcome making the model interpretable and trustworthy.

4. Identify Key Contributing Factors

To analyze and highlight the most influential features in autism prediction such as social behaviour, communication patterns, age and other factors.

3. SYSTEM ANALYSIS

System analysis is a very important phase in the development of any software system where a detailed study is carried out for the existing processes, their limitations are identified and an improved solution is designed. In the context of the proposed project, system analysis focuses on understanding the traditional methods of autism diagnosis and a need for more efficient, accurate and data-driven predictive system is evaluated.

The existing system are primarily based on clinical evaluation methods for the diagnosis of Autism Spectrum Disorder. These methods include behavioral observation, psychological assessments and standardized diagnostic tools which are administered by trained professionals such as psychologists and neurologists. Diagnosis process generally consists of structured interviews, questionnaires, social interactions, communication abilities and observation from behavioral patterns. These techniques are widely accepted and are medically reliable but these are time-consuming and need specialized expertise.

The major limitation of the existing systems is the dependency on expert professional. In many regions, especially in rural or underdeveloped areas, access to qualified scientists is limited. This is the reason for delay in diagnosis and in some cases, it is not possible to even diagnose. Early detection of autism is very critical because timely intervention can significantly improve individual's developmental outcomes. However, due to limited resources and accessibility issues a lot of individuals don't get access to personalize therapies.

Another challenge related to traditional systems is subjectivity. As diagnosis is largely based on human observation and interpretation and therefore, there are chances of variations in assessment outcomes based on the evaluator's expertise and experience. Due to this lack of standardization the consistency and reliability of diagnosis is affected.

Moreover, it becomes difficult to efficiently process the manual analysis of large volumes of behavioral data which ultimately limits the scalability.

Existing systems do not provide predictive capabilities. These mainly focus on diagnosis when noticeable symptoms appear instead of early screening which is based on subtle behavior patterns. With the increasing availability of data and advancement in computational techniques, there is also an increase in the need for systems which assist in the early prediction and screening through data driven approaches.

To address these limitations, proposed system introduces machine-learning based predictive framework which analyzes behavioral and demographic data to estimate the likelihood of Autism Spectrum Disorder. Machine learning algorithms have the ability to learn patterns from historical data and make predictions on the new data which makes them suitable for prediction of autism. By making use of structured datasets, systems identify relationships between different features and determine the impact of prediction on outcome.

Multiple machine learning algorithms are implemented and evaluated inorder to determine the most effective model for making predictions. Techniques such as cross validation and hyperparameter tuning are used to improve model performance and prevent overfitting. The system evaluates the models using performance metrics such as accuracy, precision, recall and F1-score to ensure reliable results.

A significant enhancement in the proposed system is the incorporation of Explainable Artificial Intelligence (XAI) which is implemented using SHAP (SHapley Additive exPlanations). In machine learning applications, especially in the field of healthcare, one of the biggest challenges is the lack of transparency in model predictions. Many of the models function as black boxes which provides the output but the reasoning behind is not known. It affects the trust in such system and makes it difficult for the adoption of such systems in real world scenarios.

To overcome this issue, SHAP has been used to interpret model predictions, in which each feature is assigned an importance value. The system is able to explain which factors contribute in the final predictions with this. The users can better understand the decision-making process with the results. This is very crucial especially in healthcare applications where predictions and reasoning are both important equally.

3.1 IDENTIFICATION OF NEED

Identification of need is an essential step in system development which highlights the reasons to propose a new system and justify its significance. In the context of Autism Spectrum Disorder (ASD), the need of accessible and reliable methods for early screening and prediction is increasing day by day. Autism is a complex neurodevelopmental condition which affects communication, behavior and social interaction and the impact of it is different in different individuals. Detection of it in the early stages can improve the developmental outcomes of the individual.

The main requirement of proposed system is based on the limitations analysis of the conventional methods of diagnosis. The diagnosis of autism is usually based on clinical examination carried out by experts including psychologists and neurologists. In these assessments, behavioral observations, structured interviews and standardized tests are included. The entire process is time consuming and also resource intensive. In many cases It requires multiple sessions to arrive at the final diagnosis.

The other important requirement is the problem of accessibility. The availability of qualified healthcare professionals to most regions particularly rural and underserved areas is very low. This creates a gap between the people and the diagnosis services that they can avail. Many people are not diagnosed, or are as very late which is a part of the results. This minimizes treatment and support. We can fill this gap with the help of machine learning based predictive systems since it can be used as an initial screening tool that is readily available.

Moreover, with the increasing availability of data and advancements in newer technologies, it is crucial to direct these approaches towards building a better society. The machine learning models are used to analyze large data sets in order to identify concealed patterns that would otherwise be difficult to identify when done manually.

1) **Market Analysis:** The need for AI-based healthcare solutions is increasing rapidly especially in mental health screening and early diagnosis. Autism prediction tools have a significant potential in hospitals clinics and educational institutions.

2) **User Analysis:** The primary users are mainly healthcare professionals, researchers and caregivers. The system is designed to be simple, accessible and supportive for non-technical users too.

3) **Technical Feasibility:** The project is technically feasible. It makes use of Python, machine learning libraries and standard computing resources. Tools like Scikit-learn, XGBoost, and SHAP ensure efficient implementation.

4) **Financial Feasibility:** The system requires minimal cost as it uses open-source tools and libraries. No expensive infrastructure or licensing is required.

5) **Legal Considerations:** The system must ensure data privacy and ethical use of sensitive health-related data. It should clearly state that it is a support tool and not a substitute for medical diagnosis.

6) **Market Opportunities:** There is strong potential for deployment in healthcare centers, online screening platforms, and mobile applications. It can also be integrated into telemedicine systems.

7) **Business Model:** The system can be offered as a subscription-based web application or integrated as a service for healthcare providers. It can also be scaled into a SaaS-based healthcare solution.

8) **Risk and Mitigation Strategies:** Biasness can be mitigated through proper model validation, balanced datasets (SMOTE), and continuous performance monitoring.

3.2 PRELIMINARY INVESTIGATION

Preliminary investigation is the initial phase of developing a system. It is useful for understanding the problem, to evaluate the feasibility of the proposed solution and also focusing on identifying the requirements of system implementation. In the context of the proposed project, the aim of preliminary investigation is to assess the need of an automated prediction system and to determine the practicality of using machine learning techniques to implement such solution.

The investigation started with the study Autism Spectrum Disorder (ASD), the individuals affected by it and what is the impact of it on the society. Further, the analysis highlighted that early screening of autism improves the quality of life of the individuals affected by it and with early identification, timely intervention can be made which can help in cognitive and social development of the individual.

The next step of the preliminary investigation was to explore the potential of machine learning to address such problem. Machine learning techniques are widely used in productive analysis and decision support systems in the domain of healthcare as it is good with pattern analysis, classification and prediction. It has been identified that algorithms such as Logistic Regression, Decision Tree, Random Forest, Support Vector Machine and XGBoost are suitable for solving the prediction problems related to autism screening.

In addition to prediction, the study also focused on the importance of interpretability in health care applications. It has been observed that machine learning models operate like black boxes. So, to address this challenge, Explainable Artificial Intelligence (XAI) techniques were explored. SHAP (SHapley Additive exPlanations) was identified as an effective method which assigns importance values to each feature and explain the model predictions.

During this phase, the availability of suitable data sets were also examined. Publicly available data sets such as the Kaggle repository provides structured autism screening datasets which constitute of behavioral and demographic attributes. These data sets could be used to train machine learning models but it has also been observed that these datasets consist of missing values, categorical variables and class imbalance for which preprocessing techniques such as SMOTE (Synthetic Minority Over-sampling Technique) can be used for effective model performance.

From a technical perspective, study confirmed that this project is feasible with the use of standard tools and technologies. Python programming language along with libraries such as NumPy, Pandas, Matplotlib, Seaborn, Scikit-learn and XGBoost, provide comprehensive environment for model development. Along with that, the integration of SHAP would add interpretability to the model.

In conclusion, preliminary investigation established that autism prediction is possible through automated, accessible and interpretable system. It also confirmed that Machine Learning and Explainable AI techniques can help in developing a feasible system. During this phase, the insights that are gained only helped to propose the system design and a foundation for implementation.

3.2.1 EXISTING SYSTEM

Existing systems are based on traditional clinical methods for the diagnosis and assessment of autism. However, there are a lot of limitations which point towards the need of more efficient and accessible solutions.

Currently, the diagnosis of autism is made through a combination of clinical observations, psychological assessments and structured questionnaires. Healthcare professionals such as psychologist, psychiatrists and neurologists evaluate an individual's communication skills, social interactions and behavioral patterns for a period of time. Standardized tools are commonly used for this purpose such as Autism Diagnostic Observation Schedule (ADOS) and Autism Diagnostic Interview-Revised (ADI-R). These methods are comprehensive and provide detailed insights about the conditions but they require significant amount of time, expertise and resources.

The greatest disadvantage of the current system is that, they are extremely time consuming. Diagnosis process is usually done in several sessions and through continuous observations that may postpone the diagnosis of autism. Although early detection is very crucial but due to lengthy procedures, the diagnosis is delayed. This latency influences the efficacy of therapies and support system that prevents the overall growth of the person.

Dependence on professional expertise is a limitation on its own. In the diagnosis process, special knowledge and experience is needed that is not present in all regions of the country. Underdeveloped or rural regions do not have qualified healthcare providers hence the access is restricted.

Over the past few years, there are hardly any new digital tools and screening apps to measure autism. Such systems typically take the form of questionnaires or rudimentary rule-based solutions to offer initial information. In a way, it enhances the accessibility but they are not highly analytical and precise. Most of the tools do not use machine learning techniques so they are not able to effectively capture the complex relationship between the data.

Moreover, the existing systems do not possess any predictive capabilities. They mostly do the diagnosis of autism when there are noticeable symptoms. With the use of such systems, predicting the likelihood of autism at an earlier stage is not possible.

The other limitation is that, the sophisticated computational models are not interpretable. Some modern systems which incorporate machine learning, do not provide clear explanations for the predictions that were made. The absence of transparency influences the trust and, in particular, in healthcare applications.

3.2.2 PROPOSED SYSTEM

The proposed system is designed, keeping in mind the limitations of traditional approaches. It aims to provide a data-driven, efficient and interpretable solution which helps in early screening of autism. The system utilizes the machine learning techniques by analyzing behavioural and demographic data of the individuals who may have the likelihood of having autism.

The architecture of the proposed system is modular and it consists of some key components such as data acquisition, data preprocessing, exploratory data analysis, model training, evaluation, prediction and explainability. Initially, the system accepts a structured data set which consists of various features or attributes such as behavioural scores, age, gender and other demographic attributes. This dataset is then processed to match quality and consistency. Data preprocessing techniques such as handling missing values, encoding categorical variables and balancing classes is also applied so that data can be made available for effective model training.

When the data has been pre-processed, multiple supervised machine learning algorithms are implemented such as Logistic Regression, Decision Tree, Random Forest, Support Vector Machine and XGBoost. These models are trained using the data set and assessed based on performance metrics like accuracy, precision, recall and F1-score. Cross validation and hyperparameter tuning techniques are used to improve the model and ensure proper generalization and based on those evaluation metrics, the best performing model selected.

In order to solve the problem of black box nature of AI, SHAP is built into the system where clear and understandable explanations are provided. SHAP operates by giving the importance values of each feature that demonstrates the degree to which each factor participates in the final forecast. It presents global and local explanations which demonstrate the overall significance of features to the dataset and respectively, specific predictions to particular cases.

For example, when it comes to predicting autism, SHAP can reveal which behavioral scores or demographic traits have the strongest impact on the outcome. It is a graphical depiction of whether a given feature enhances or reduces the probability of autism. This enables the user to not only be aware of what the prediction was but also the reasoning behind it.

The SHAP application promotes transparency, and reliability of the system. This has significance primarily in the field of healthcare where the decisions are to be explainable and justifiable. The system predictions are more confident to be trusted by healthcare professionals and users.

Advantages:

1) **Early Detection Support:** The system helps in early screening of Autism, enabling timely intervention and better developmental outcomes.

2) **Data-Driven and Objective:** Unlike traditional methods, the system reduces human bias by using machine learning algorithms.

3) **Explainability using SHAP:** The integration of SHAP provides clear explanations for each prediction, removing the black-box nature of AI and increasing trust.

4) **Time Efficient:** The system generates predictions quickly, reducing the time required compared to lengthy clinical diagnostic procedures.

5) **Cost-Effective and Accessible**: Built using open-source tools, the system can be deployed easily and used in areas with limited access to healthcare professionals.

Disadvantages:

1) **Not a Replacement for Medical Diagnosis:** The system can only assist in screening and cannot replace professional clinical diagnosis.

2) **Dependence on Data Quality:** The accuracy of predictions depends heavily on the quality and completeness of the dataset used for training.

3) **Limited Real-World Validation:** The model is trained on available datasets and may not generalize perfectly to all real-world scenarios.

4) **Handling Sensitive Data:** The system deals with personal and behavioral data.

3.3 FEASIBILITY STUDY

For the proposed project, feasibility analysis makes sure that the system can be successfully developed and implemented using available resources and technologies.

The main goal of the feasibility study is to check if the system can solve the problem properly or not. It also needs to be accurate, reliable, and easy to use. This project uses Machine Learning along with Explainable AI to give useful predictions. So, it is important to make sure that the system works well with current technology and is also practical to use in real life.

This technically enabled system is possible to implement as it is built on Python a common programming language, along with simple tools. Tools like Scikit-learn and XGBoost assist the system to predict. SHAP helps elucidate how those predictions are formed. All of these tools are stable and simple to understand. This ease is made possible by all of these. Furthermore, unique or costly hardwares are not required, the system can run on a simple computer.

Economic viability is another important aspect of this study. The entire project exclusively relies on open-source tools and platforms, making it cost-effective. It does not involve any expensive licenses or infrastructure and the system can simply be developed on a personal computer. While this makes the project assesses to contribute as financially viable, it does work well on academic environment. Also because it is economical, it can be scalable in the future and potentially deployed to a real-world application.

The operational feasibility focuses on how much of the system is effective for the intended users. The proposed system has been kept simple, user-friendly and accessible. It accepts input data in a structured format and also provides explanation with predictions so that it becomes easier for the non-technical users to understand what is going on.

Legal and ethical compliance is crucial in this case perhaps more than anything else because this system handles sensitive health data. Therefore, confidentiality is not a mere formality. It is embedded in the operation of the system. The data is anonymous and appropriate safeguards are in place to ensure its secure handling. At the same time, the system is not intended to act as a physician. It is an auxiliary tool, something that can complement, not replace, professional medical judgment. This distinction helps to avoid many ethical gray areas. This approach is efficient in maintaining the accountability without compromising on the efficiency of the system.

Moreover, some strengths and weaknesses as well as some threats and opportunities, have been analyzed. These range from poor data quality to model bias and prediction errors. However, these issues can be overcome by thorough data cleaning, the use of balanced data, and verification of the proper functioning of the model.

Doctors, researchers, and even the general public can use the system to detect contraband. A detailed design study confirms that the system can be built and used in practice. It is compatible with existing technology, cost-effective, user-friendly, and ethically responsible.

3.3.1 TECHNICAL FEASIBILITY

In the proposed project technical feasibility plays a crucial role to make sure smooth development and deployment of the project is done.

The system is developed using Python, which is one of the most widely used programming languages in the field of data science and machine learning. Python supports simplicity, flexibility and provides a rich ecosystem of libraries such as NumPy and Pandas which are used for efficient data handling and manipulation, while Matplotlib and Seaborn are used for data visualization. Machine learning models are implemented using Scikit-learn and XGBoost, which provide robust and efficient algorithms for classification tasks.

An important key component of this project is the integration of Explainable Artificial Intelligence (XAI) which is implemented using SHAP. SHAP is a powerful library which provides interpretability to the machine learning models and it is technically feasible because it is compatible with tree- based models such as Random Forest and XGBoost which are used in this project.

The development environment of the project is Jupyter Notebook. It supports interactive coding environment, debugging tools and visualization which makes the development process efficient and smooth. The system is designed to run on a local machine so it doesn't require any complex deployment infrastructure.

From the hardware perspective, the system does not require any high-end computing resources. A standard computer which has minimum 8GB RAM and a modern processor which can sufficiently handle data preprocessing, model training and prediction tasks. This makes the system accessible and easy to implement without any additional investment.

The dataset used in this project is in CSV format which is easier to handle and it is compatible with different python libraries. Data preprocessing techniques

such as handling missing values, encoding categorical variables and class balancing is supported through the existing libraries. Apart from this, the integration of SHAP enhances the system capability for providing explanations to the predictions without introducing any significant technical complexity.

Component	Description
Operating System	Windows 10 / Windows 11 / Linux
Programming Language	Python 3.x
Libraries	NumPy, Pandas, Matplotlib, Seaborn, Scikit-learn, XGBoost, SHAP, Imbalanced-learn (SMOTE), Pickle
Development Environment	Jupyter Notebook / Visual Studio Code
Dataset Format	CSV (Structured Dataset)
Platform	Local Machine Execution
Hardware Requirement	Minimum 8GB RAM, Intel i5 Processor or above
Machine Learning Techniques	Supervised Learning (Classification), Cross-validation, Hyperparameter Tuning
Explainability Tool	SHAP (SHapley Additive Explanations)

Table No.3.3.1- Software Dependencies Used in the Project

3.3.2 ECONOMICAL FEASIBILITY

Economic feasibility Is related to evaluating the financial aspects of this proposed system so that it can be easily determined if the system is cost effective and financially viable. It consists of analyzing costs related to the system development, implementation and maintenance. Along with that, potential benefits are also evaluated that the system can provide. For this proposed project, economic feasibility is an important consideration specially because this system is developed as a part of an academic and industrial training initiative.

The proposed system is highly cost effective because it mainly uses open-source tools and technologies. The cost related to data acquisition is also very minimal. The dataset used in the project is publicly available on sources such as Kaggle which does not require any costly data collection process. However, with further enhancements of the project, primary dataset can be collected and generated and the machine can be trained on it.

From the perspective of maintenance, the system is kept simple and manageable because it can easily operate on local machines with the use of stable technologies so, the cost of maintenance becomes negligible. Any updates or improvements can be made easily without any significant financial investment.

In terms of benefits, the system can add value in the process of early screening of autism and with the early detection, developmental outcomes can be improved. The long-term cost for therapy and support can be reduced significantly. The system is not for replacing professional diagnosis but, it can serve as a tool for preliminary screening so that the healthcare professionals face lesser burden and optimize the utilization of resources.

Moreover, the system also has a potential for future scalability. With relatively low additional cost, this system could be developed into a web based or mobile application in order to become accessible to larger population. This opens the opportunities for health care centers, educational institutes and telemedicine

platforms which can again enhance its economic value.

The proposed system is economically feasible because the developmental and operational costs are low and it uses open-source technologies with minimal hardware requirements. Whatever benefits are there for early screening and improve accessibility, this system is able to provide them at a very minimal cost. This makes the system a practical and financially viable solution for the prediction of autism and to address mental health awareness.

3.3.3 OPERATIONAL FEASIBILITY

Operational feasibility refers to how much of the proposed system can be effectively implemented in an organization or by the intended users. It evaluates whether the system can smoothly function in real world environment or not, can meet user requirements or not and will it be accepted by the stakeholders or not. For the proposed project, operational feasibility is an important aspect as it is designed to assist in the early screening of autism.

The key strength of the system is its user-friendly design. The workflow of the system is straight forward which starts with data input, then processing, prediction and result interpretation. It ensures simplicity so that healthcare professionals, researchers and caregivers can use it easily without having any advanced technical knowledge. System is designed in a way to minimize complexity and provides meaningful results in an efficient manner.

Moreover, the integration of Explainable AI (XAI) with the use of SHAP makes it easier for the person using it because it not only generates the results but, also clarifies the reasoning behind it. For example, the system can show which factors contributed to a higher or lower autism risk. This way the output becomes more informative and actionable.

Integration in workflows is thus another important aspect which contributes to the system's operational feasibility. This proposed System is developed in the form of a supportive tool that can easily be integrated into existing processes in healthcare and research environment without interfering with them.

There is little to no training for the user to use this system. So the functionality of this is very simple so you can easily know how to use the system. Nothing fancy, a simple idea on inputting data and interpreting the result is enough. Other than that, it also incorporates some visual outputs like graphs and charts which can help provide a better understanding of the system enhancing engagement.

To ensure reliability of the system which also consists of consistent performance is done by validated machine learning methods e.g. cross validation/model evaluation metrics. With this the chances of incorrect predictions become very low and the user gains confidence on the system. Few others preprocessing techniques such as SMOTE help in class balancing to improve performance of the model.

3.4 PROJECT PLANNING

Project planning is a critical phase in the development of any system because it defines a roadmap for successful implementation. It consists of organizing tasks, allocating time, identifying dependencies and ensuring that the progress should be made in a structured and systematic manner. For the proposed project, project planning is very essential in order to ensure that learning phase and the development phase are both completed under the given timeline efficiently.

This project is completed in the duration of two months as a part of an industrial training program. Planning has been divided into two major phases: learning phase and project development phase. This approach ensures a strong foundation has been built before implementation.

The focus of the learning phase is to acquire required skills and knowledge for the project. In this phase, python programming, data structures, object-oriented programming, data analysis, statistical concepts and machine learning techniques were covered. Libraries such as Numpy, Pandas, Matlplotllib, Sea born and Scikit-learn are studied in detail. In this phase, key concepts such as supervised learning, model evaluation, cross validation and ensemble method were studied and understood.

Project development phase start after the completion of learning phase. In this phase, the practical implementation has been done for autism prediction system. It starts with problem definition and the selection of dataset. After that, data cleaning and preprocessing is done. After the preparation of data, Exploratory Data Analysis (EDA) has been performed in order to identify patterns, relationship and feature distributions. With the use of visualization techniques, insights from the dataset are found. Then the moderl training phase begins where multiple machine learning algorithms such as Logistic Regression, Decision Tree, Random Forest, Support Vector Machine and

XGBoost are implemented. These models are then evaluated through performance metrics such as accuracy, precision, recall aur F1-score.

After this, an important component that is the integration of Explainable Artificial Intelligence (XAI) is done in order to ensure that the model not only does predictions but explain the reasoning. Planning also includes hyperparameter tuning and cross validation for the model in order to improve the performance and ensure proper generalization. After the selection of best performing model, the system moves towards prediction phase where new input data is processed to generate predictions.

1. Project Goals and Objectives:

- To develop a machine learning-based system that can predict the likelihood of Autism using behavioral and demographic data.
- To implement and compare multiple classification algorithms such as Logistic Regression, Decision Tree, Random Forest, SVM, and XGBoost to identify the best-performing model.
- To integrate Explainable Artificial Intelligence (XAI) using SHAP in order to interpret model predictions and generate reasoning.
- To make a screening tool that is user friendly and reliable for early detection of autism.

2. Detailed Project Plan:

- Consists of collection of dataset and then making it ready for further processes by handling any missing values and different techniques as required.
- Implement and test different machine learning algorithms in order to find the best suited one for the project.
- Once the model is capable of generating predictions, integrate techniques for model interpretability and complete documentation and reporting.

3. Development Platform:

- The project is developed using Python 3.x as the primary programming language due to its simplicity and strong support for machine learning libraries.
- The development environment used is Jupyter Notebook and Visual Studio Code, which provide interactive coding, debugging, and visualization capabilities.
- Various libraries such as NumPy, Pandas, Matplotlib, Seaborn, Scikit-learn, XGBoost and SHAP are used for data processing, model building and explainability.
- The system is executed on a local machine platform (Windows/Linux) with standard hardware requirements, ensuring easy development and testing without complex infrastructure.

4. Safety and Security Measures:

- The system uses a publicly available open-source dataset for training and testing purpose.
- Input validation is also introduced such that the results remain errorless and correct.
- The project and associated data files are stored using pickle with restricted access.
- The system abides by the ethical guidelines, serving as a supportive tool and not a replacement to medical diagnosis.

5. Legal and Security Measures:

- As this system acts as a supportive tool it reduces legal risks.
- The system avoids storage of sensitive personal information following data privacy policy.
- Proper data handling mechanisms are also implemented to store the project and protect it from unauthorized accesses.

6. Testing and Launch:

- Proper testing at all phases has been done to ensure verify the system.
- Performance metrics have been used to evaluate the working of the system.
- The system is deployed on a local environment for practical use.
- It has a scope for future web or cloud-based deployment.

7. Marketing Strategy:

- The system can be promoted as screening tool for early detection of Autism which can support clinicians and medical staffs.
- Awareness in people about the project as well as autism can be made and promoted through seminars, webinars or campaigns.

3.5 PROJECT SCHEDULING

Project scheduling is an important component of project management which plans and organize tasks in a defined timeline. It ensures timely and efficient completion. It helps in resource allocation, identifying dependencies and monitor progress during the developmental process. For the proposed project, project scheduling plays a crucial role in managing activities as a part of industrial training project for the period of two months.

Project scheduling for this project has been done keeping in mind all of the phases in the learning phase as well as in the developmental phase The final stage of the schedule includes testing validation and documentation All of the modules are then tested Correct functionality can be ensured and results can be consistent and reliable in the documentation phase The project report has been prepared which consists of all of the findings visualization and explanations.

Project schedule also considers task dependencies and ensures the activity is completed before it moves to the next stage. It helps in avoiding monitoring delays by allocation of proper time and maintains workflow efficiency. Overall, project scheduling is well organized and structured in order to ensure smooth execution.

Task	Phase	Duration (Days)	Start Date	End Date	Dependencies
A1: Internship Orientation & Program Introduction	Learning	1	2026-02-06	2026-02-06	None
A2:Python Fundamentals	Learning	5	2026-02-07	2026-02-11	A1
A3:Data Structures & File Handling	Learning	4	2026-02-12	2026-02-15	A2

Task	Phase	Duration (Days)	Start Date	End Date	Dependencies
A4:Object-Oriented Programming	Learning	3	2026-02-16	2026-02-18	A3
A5:Data Analytics and Visualization	Learning	7	2026-02-19	2026-02-25	A4
A6:Statistics for Machine Learning	Learning	5	2026-02-26	2026-03-02	A5
A7:Machine Learning Concepts	Learning	5	2026-03-03	2026-03-07	A6
A8:ML Algorithms Implementation	Learning	6	2026-03-08	2026-03-13	A7
A9:Mini Assessments & Practice	Learning	3	2026-03-14	2026-03-16	A8
B1: Project Topic Finalization	Project Development	2	2026-03-17	2026-03-18	A9
B2: Dataset Collection & Understanding	Project Development	3	2026-03-19	2026-03-21	B1
B3: Data Cleaning & Preprocessing	Project Development	6	2026-03-22	2026-03-27	B2
B4: Exploratory Data Analysis	Project Development	4	2026-03-28	2026-03-31	B3
B5: Model Training	Project Development	5	2026-04-01	2026-04-05	B4
B6: Hyperparameter Tuning	Project Development	3	2026-04-06	2026-04-08	B5

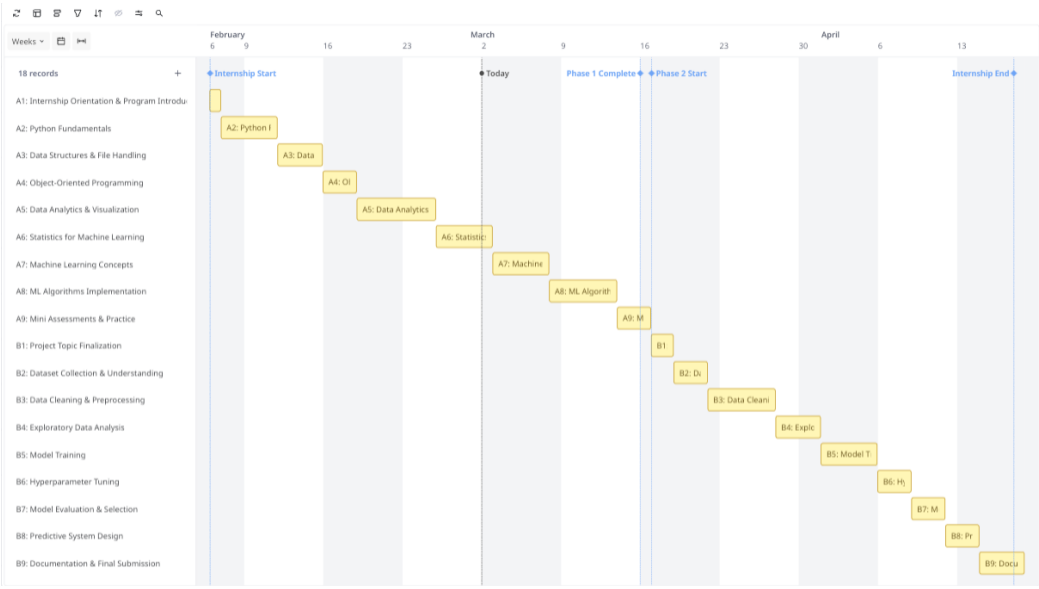
Task	Phase	Duration (Days)	Start Date	End Date	Dependencies
B7: Model Evaluation & Selection	Project Development	3	2026-04-09	2026-04-11	B6
B8: Predictive System Design	Project Development	3	2026-04-12	2026-04-14	B7
B9: Documentation & Final Submission	Project Development	4	2026-04-15	2026-04-18	B8

Table No. 3.5.1: Gantt-Based Schedule of Learning and Project Activities

3.5.1 GANTT CHART

The Gantt chart provides a visual representation of the project schedule which illustrates the sequence of activities, their duration and dependencies throughout the development of the project. It ensures timely completion of the project in a given time frame with effective planning and progress monitoring.

The project is divided into two major phases: learning phase and developmental phase. Learning phase consists of all of the foundational tasks that are included in order to build skills and knowledge for the implementation phase or the development phase. So, from the starting of the phases to the project implementation it helps in keeping a track of activities.



Their duration and dependencies throughout the development of the process Fig no. 3.5.1.1: Gantt chart

3.5.2 PERT CHART

The Program Evaluation and Review Technique (PERT) chart is an important project management tool which is used to analyze and represent the sequence of tasks involved in the development of a system. It helps in identifying task dependencies, estimating timelines and to determine required critical path for successful completion of the project. For the proposed project, PERT chart helped in efficiently organizing and managing the time period.

In the PERT chart, every activity is represented in the form of a node and the connections between those nodes indicate the dependencies. The flow starts from the learning phase to understanding he basics and fundamentals. Moving to advanced concepts of machine learning and how to implement those in the developmental phase such as data collection, preprocessing, EDA to hyperparameter tuning. Every task depends on the completion of the previous task which forms a clear logical progression.

The following PERT chart systematically represents the entire workflow of the project which includes the sequence of the task dependencies and timelines. The project started from 6th February, 2026 where the initial activities such as ML foundation, design and learning modules started. After that, data analytics and statistics were learned which provides foundation for data preprocessing. Following that, a practice of ML algorithms and model training was also done in the early phase. Then, before the project development phase started, literature review and data set collection activities were also included. These activities provide input for Exploratory Data Analysis (EDA). In the final stage, documentation and final review is done to ensure proper project completion.

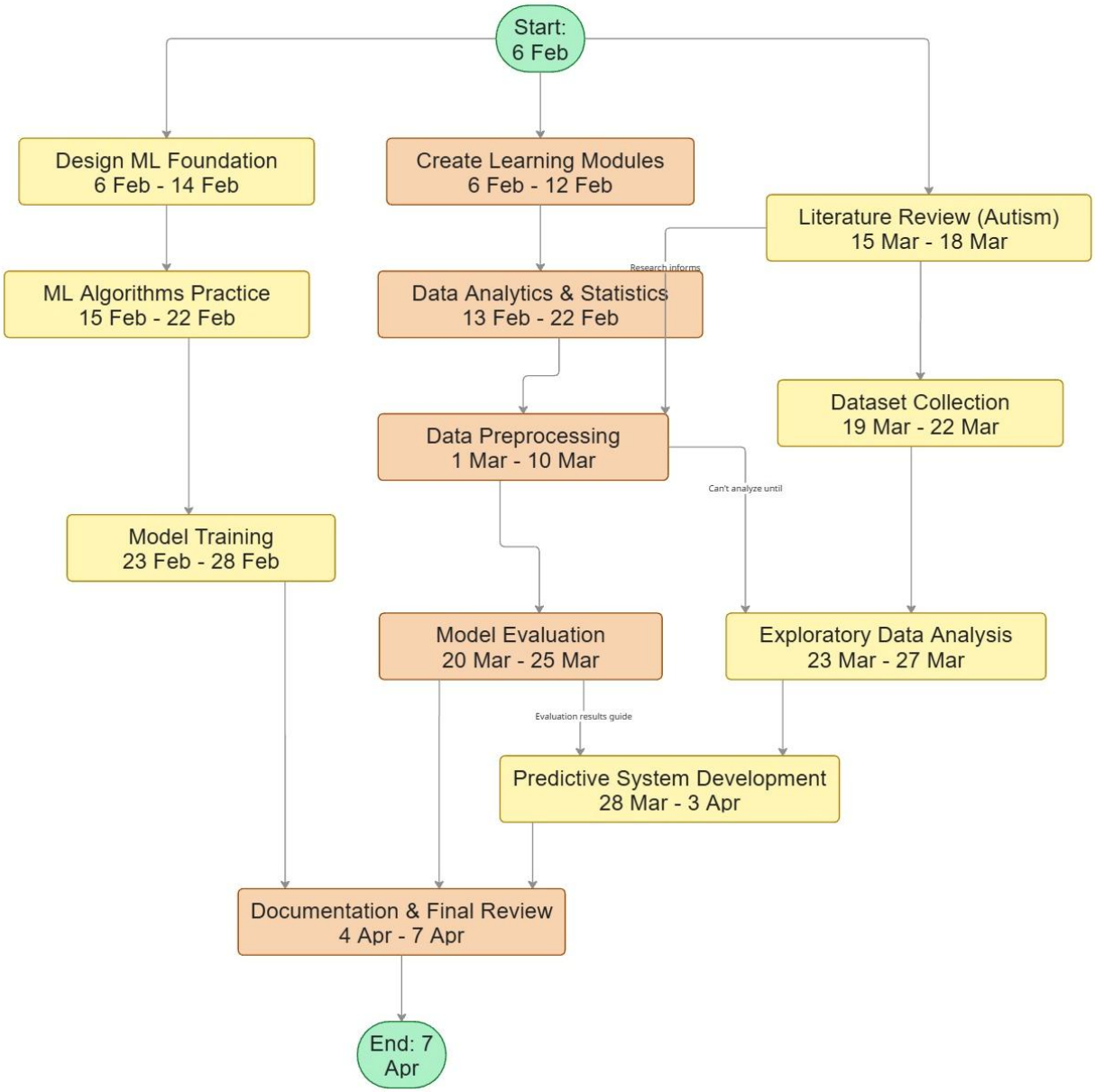


Fig 3.5.2: PERT chart

3.6 SOFTWARE REQUIREMENT SPECIFICATION

Introduction

For this project, SRS outlines the requirements for data processing, model training, prediction and explainability. It ensures that all of the stakeholders clearly understand the objectives and functionalities of the system, ensuring a reliable and user-friendly predictive system is designed, developed and implemented.

Purpose

The purpose of this system is to design and develop an efficient and reliable machine learning based tool which utilizes behavioral and demographic data in order to predict autism in individuals. This system uses SHAP for transparent and interpretable results which supports early screening in order to assist healthcare professional for making informed decisions.

Scope

This project focuses on developing a machine learning based system which utilizes structured data to predict autism in individuals. It consists of data preprocessing, model training, evaluation and explainability. The system is designed for early support for screening of autism and in future, it can also be extended into web or mobile platforms to increase accessibility to wider audience.

Benefits

The system provides support for early screening of autism which allows for timely intervention for better development of social and mental health of the individual. It gives accurate and data driven predictions, reducing human bias and the integration of explainability improves transparency and interpretability, increasing the trust of users.

3.6.1 Functional Requirement

To accomplish this project, we must satisfy the following functional requirements:

1) The system shall load and process a dataset in CSV format and display basic information such as shape, data types, and summary statistics.
2) In order to make unbiased and correct predictions, the system should first evaluate then perform data preprocessing techniques on the dataset.
3) The system should train different machine learning models and choose the best one according to the performance.
4) The system should also provide explanations with the output in order to create a better understanding of the diagnosis.

3.6.2 Non-Functional Requirement

1) The system should achieve a high accuracy score for better reliability.
2) The system should be able to generate correct response in minimal wait time.
3) The system shall maintain data security and privacy. It should ensure that the sensitive data of the user is handled correctly.
4) The system should be user friendly and requires minimum training to use it.

3.7 SOFTWARE ENGINEERING PARADIGM APPLIED

The project follows incremental software engineering paradigm. Incremental model is a famous approach where the software is built and delivered in small increments or manageable parts. Each increment or part adds new functionalities to the project.

The reason of following this paradigm is that, this system is builts in multiple stages such as collection of datasets, applying data preprocessing techniques on the dataset, training different models for prediction and so on. Each stage can be treated as a separate module and developed independently after the current stage is completed.

In the initial phase, the basic functionalities were added to the project which consists of data collection, preprocessing and model training. Furthermore, as the system is trained and verified by testing and evaluating, more functionalities are added to the system step-by-step. Hyperparameter tuning, cross validation and the integration of explainability are added in the next increments.

The main advantage of using this particular paradigm is flexibility. It allows to be flexible with the updates or changes made to the system without disrupting any previous processes. For example, different models can be trained and tested simultaneously and the best one can be chosen out of it. Other than that, more functionalities can be added gradually to the system without causing any disruption to the core functionalities.

Another advantage is the benefit of risk reduction. As the project is built in small increments, it becomes easier for the developer to find and reduce errors in the initial stage rather than completing the entire system development. This helps in identifying risks way earlier in the process by performing continuous evaluation at each step.

The incremental paradigm also helps in better utilization of resources and time management. As all of the phase are divided in the said amount of time, it becomes easier to meet deadlines and focus on the phase of the developmental process. This is very crucial in any organization or academic setting.

1. Agile Development Approach

- Agile Development approach has been followed during the developmental process as continuous testing of each module is done and constant feedback was incorporated at each step.
- Following this approach added flexibility in the process as based on the requirements or the evaluation of performance of the system, updates were made.

2. Object-Oriented Programming (OOP)

- The system follows OOP principles such as abstraction, showing what is necessary to the user and encapsulation where different modules are treated together as one entity.
- Reusability is achieved as proper file storage and handling is done through pickle files where the code is stored and save with restricted access in order to be referenced for future use.
- The use of OOP principles added modularity and reusability to the code making it less hectic to manage and reuse.

3. Continuous Integration and Continuous Deployment (CI/CD)

- As a part of Agile developmental process, continuous feedback is taken into account which allowed for time-to-time integration of different functionalities or updates made to the system as per the requirement.

- The systems possess a potential to be easily integrated or incorporated into a web or mobile app or software framework.

4. Microservices Architecture

- The system is developed following a modular approach where each phase of the system is built in small increments.
- Each of these modules can be build, tested and updated independently allowing for easy maintenance of the system.

Tools/ Platform Used

Software Requirements Being Used:

- **Programming Language:** Python
- **Libraries:** NumPy, Pandas, Matplotlib, Seaborn, Scikit-learn, XGBoost
- **Development Environment:** Jupyter Lab
- **Dataset Format:** CSV

Hardware Requirement:

- Processor: Processor: Intel i5 or above
- RAM: 8GB minimum
- Storage: 256GB SSD
- Internet connectivity

Software Requirement:

- Processor: Operating System: Windows 10 / Linux
- Programming Language: Python
- IDE: Jupyter Notebook / VS Code
- Platform: Local Machine Execution
- **Libraries and Tools: NumPy (Numerical computations)**
    - Pandas (Data manipulation and analysis)
    - Matplotlib & Seaborn (Data visualization)
    - Scikit-learn (Machine Learning algorithms and evaluation)

    - Imbalanced-learn (SMOTE implementation)
    - XGBoost (optional advanced boosting algorithm)
    - Pickle (Model serialization and saving)
- **Machine Learning Techniques:**
    - Cross-validation (K-Fold)
    - Ensemble learning methods
    - Bias-variance tradeoff management
- **Performance Expectations:**
    - Balanced evaluation metrics considering class imbalance
    - Generalizable model with reduced overfitting
    - Reliable predictive performance suitable for screening assistance

3.8 DATA MODELS

3.8.1 FLOW CHART

The flowchart in Fig No.3.8.1 shows the step-by-step approach of the execution of the development of the project. It helps us to understand the sequence of the tasks carried out in the entire process and the data flow. Sometimes, it can also be referred as a guide for proper implementation of the program.

The developmental process starts with loading the data set which is represented just after the start node. I n this phase, the dataset with required features is loaded as a part of the project. The dataset used here is in CSV format as this format is suitable to work with different machine learning algorithms.

The next step is to clean the dataset and make it ready to use for the further phases of the development process. The system identifies the problems with the dataset such as finding inconsistencies in the dataset, missing values, class imbalance and so on. Accordingly, data preprocessing techniques are applied to make the dataset clean and useful.

After the second step, Exploratory Data Analysis (EDA) is performed on the dataset where we explore the data through visualization in order to find patterns and distributions. Histograms, boxplots and correlation heat maps are generated as a part of this step and analysis is made as to what steps are to be performed next.

The system moves toward training and testing. This step is essential for evaluating the model's performance. Training is used to train the model on the dataset while testing is used to validate the model's performance. Multiple machine learning algorithms such as Logistic Regression, Decision Tree, Random Forest, Support Vector Machine aur XGBoost are implemented.

Each model is trained and evaluated through matrix Such as accuracy, precision, recall aur F1-score. Next, the performance is compared for all of the models and the best performing model is selected. This ensures that we select the most accurate and reliable model for prediction.

Another important part of the flow chart is the integration of Explainable AI which is able to provide both global and local explanations. It also shows how each feature contributes towards the prediction. Next, the system moves towards the prediction stage where user inputs new data and the system makes use of the trained model to process the input and generates a prediction for the likelihood of autism in the individual.

Finally, the process displays the output and completes on the end node in the flowchart where prediction results and explanation are presented to the user. The flowchart ensures that all of the steps are logically connected and systematically executed.

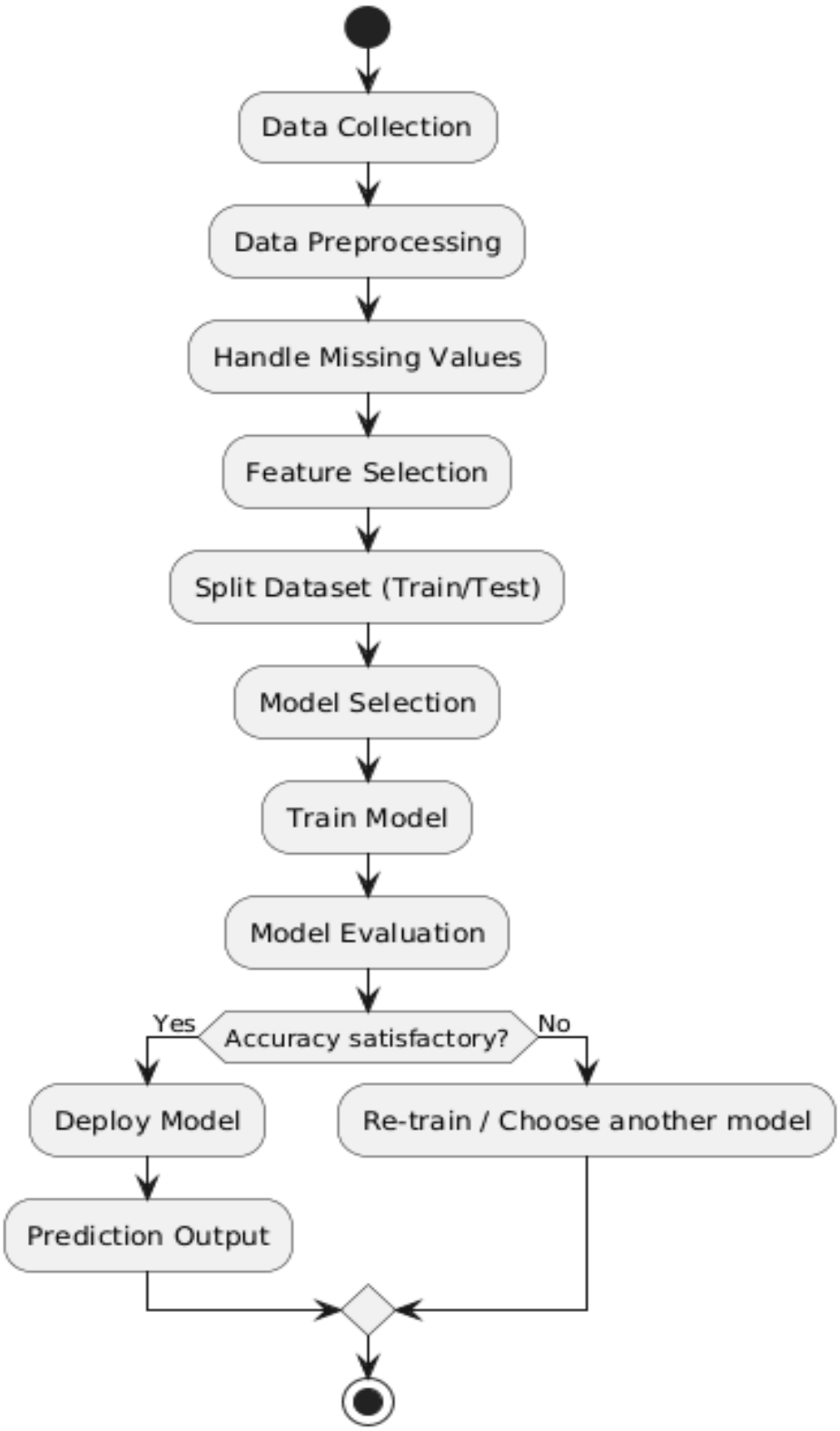


Fig No.3.8.1: Flow Chart

3.8.2 SEQUENCE DIAGRAM

The sequence diagram can also show interaction with different objects. However, its main purpose is to show how the data flows through the system. The sequence diagram for this project shows the step-by-step execution process starting from user input until producing a final prediction along with the explanation.

The sequence begins with the user who is going to interact with the system. He is the primary actor. Now, the user provides input in the form of behavioral and demographic data such as screening scores, age, gender or other relevant attributes. This input goes to the input module which is the first interaction between the user and the system.

Once the input is received, the system then forwards the data to the Data Preprocessing Module. This module is responsible for preparing the input data for prediction. It is responsible to perform tasks such as handling missing values, encoding categorical variables into numerical form, scaling and normalizing the data if required. The preprocessing module also ensures that the input data follows the same format that was followed during model training.

After preprocessing, the data is passed to the Model Loader Module. It loads the pretend machine learning model from the storage This model is already developed, trained and optimized during the model training phase. After the model is loaded, preprocessed input data is sent to the prediction module. This module applies machine learning model to input data and generates the prediction. The prediction is forwarded for interpretation to the next stage.

The next step is very crucial that is, the explainability module where the interpretation is generated for the users to better understand why the prediction has been made. Explainability module generates visual outputs such as summary plots, bar graphs and force plots which clearly shows the

feature importance and impact on the predictions. These explanations are then passed to the output modules where the predictions are combined with the results.

Through the entire sequence, each module interacts by following an order so smooth communication and data flow is achieved. Sequence diagram also highlights the modular nature of the system where each component has a specific function to perform and how they communicate with different components.

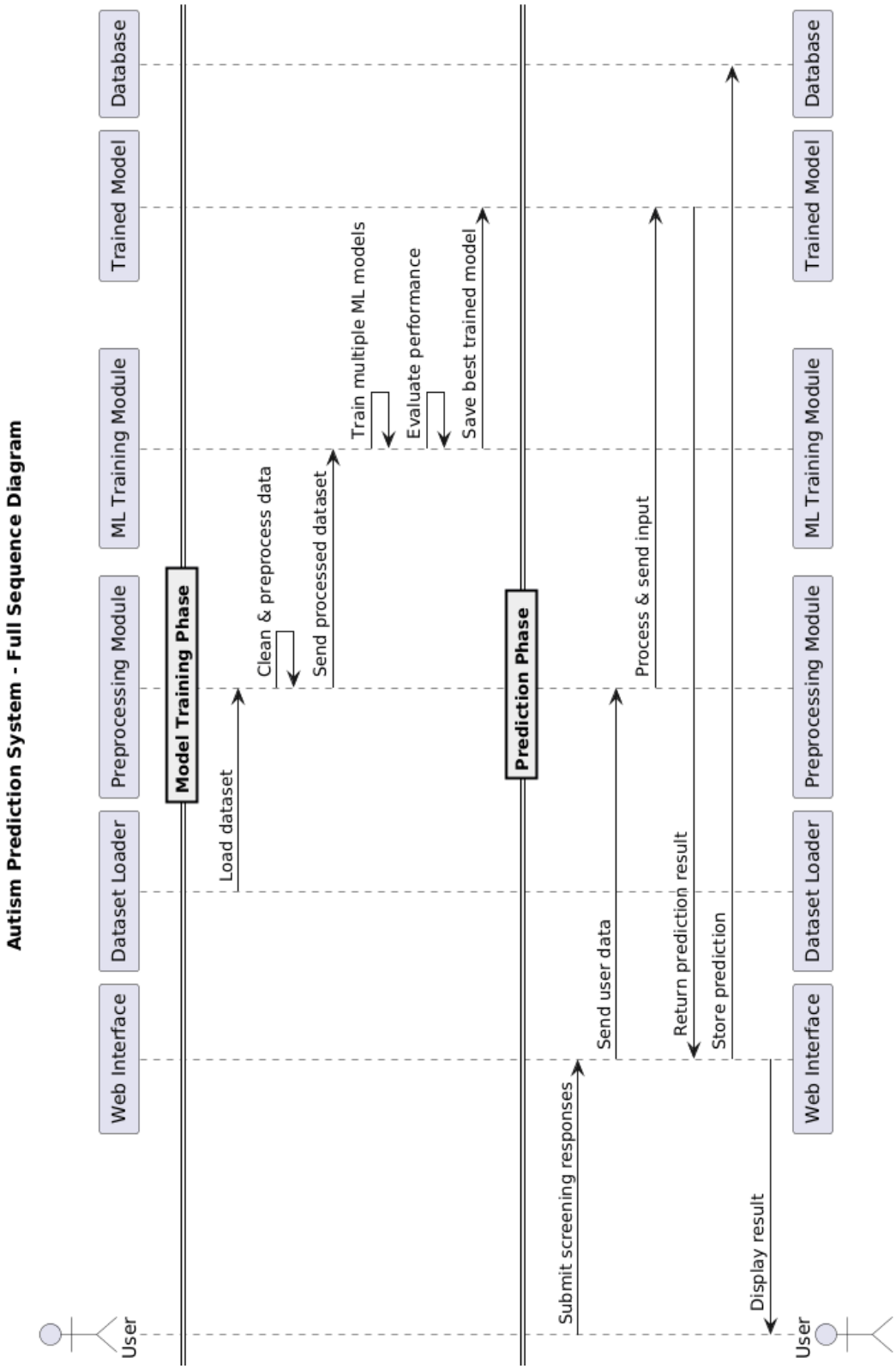


Fig No.3.8.2: Sequence diagram

3.8.3 Activity Diagram

The activity diagram describes the workflow of the system as it shows the sequence of activities that are involved in the execution of the project. It gives a clear picture on how various operations are performed with decision points, parallel processes and data flows from the start and finish. The activity diagram assists in visualizing the overall system operation and makes sure that all processes are logically related.

The process starts with the first node (start), which is the start of the system. Upon the start of the system, the initial operation is data input or dataset loading. The data set that includes the demographic and behavioral features are loaded into the system in this step.

Once the data has been loaded, the system moves on to the data validation activity. During this step, the system verifies the non-existence of values, the wrong structure, and the improper consistency of the dataset. At this point, a decision node is added to decide whether the data is valid or not. In case of incomplete data or errors, the system takes corrective measures like dealing with missing values or soliciting valid input. In case data is valid, workflow proceeds to the next step.

Data preprocessing is the next activity and one of the most important steps of the system. This involves cleaning of the dataset, coding of categorical variables into numbers and identification and dealing with outliers. This makes the data ready on which the machine learning models will train.

After preprocessing, the system moves to the exploratory data analysis (EDA) phase. The tasks associated with this phase are plotting graphs, creating correlation heatmaps and calculating the feature importance. The step assists in determining the major patterns and insights that determine the prediction.

The system then moves to the data splitting process after EDA, and the data is

separated into training and testing. It is a significant procedure in assessing machine learning model performance. The training dataset is used to train the model, while the testing dataset is used to validate its performance.

Model training is the next significant task, during which a variety of machine learning algorithms are applied. The training dataset is used to train these models. After this, model evaluation is done in the system. A decision node is employed to pick the most performing model in terms of the following evaluation metrics.

After picking the best model, the system goes to the integration of Explainable Artificial Intelligence (XAI) with SHAP. This works with the interpretation of the model predictions and finding out which feature contributes the most.

This system is then transferred to the prediction activity, in which the user inputs new data. The training data and the input data are preprocessed in a similar way and there should be consistency between the two. The trained model is then applied to produce the prediction that represents the probability of the autism.

The system carries out result interpretation after prediction. SHAP explanations are produced to offer insights on the prediction. This involves finding out what features have positively or negatively influenced the outcome. The descriptions are in visual form in the form of bar plots and summary plots.

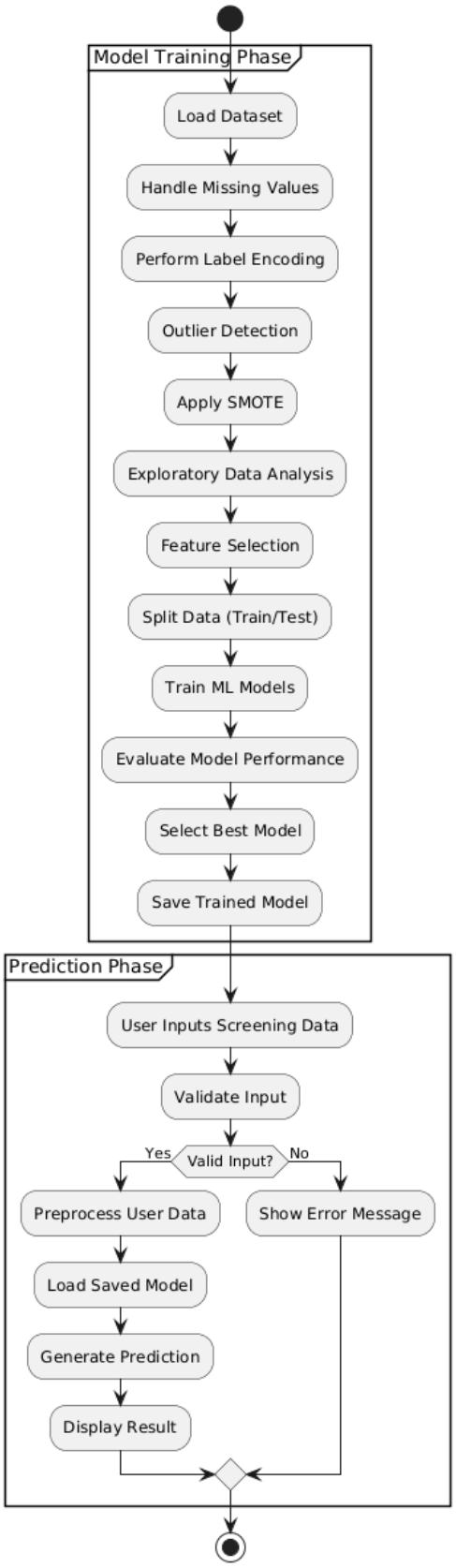


Fig No.3.8.3: Activity Diagram

3.8.4 ER Diagram

Entity Relationship diagram or ER Diagram is a conceptual representation of the data structure of the system. It defines the entities involved in the system, their attributes and the relations between them. ER diagram plays an important role in making us understand how data is organized, stored and processed in a system.

As the system primarily deals with behavioral and demographic data for the prediction of autism in individuals, the ER diagram consists of several key entities such as User, Input Data, Dataset, Model, Prediction and Explanation with each having specific attributes and relationships.

The first entity is the User, who represent the individuals interacting with the system. The user can be a healthcare professional, researcher or caregiver who inputs data into the system. The attributes of this entity consist of User_ID, Name and Role. The user is responsible for providing input data and receiving prediction results. The relationship between the User and input data is one-to-many.

The second entity is Input Data. It contains the information provided by the user. It includes attributes such as Age, Gender, Screening Scores (A1 to A10) and other behavioral features. Each input record is unique and is identified by an Input_ID. This entity is directly linked to the Prediction entity, as each input leads to a specific prediction result.

Another important entity is Dataset which represents data that is used to train and test the models. This dataset is sourced from External sources such as public repositories. The entities in this attributes include Dataset_ID, Source, Number_of_Records and Feature_List. The relationship of data entity is with model entity because the dataset is used for training and evaluating the model.

The Model entity represents all the machine learning algorithms used in the model. The attributes of this entity are Model_ID, Model_Type, Accuracy,

Precision, Recall and F1-Score. The relationship between Model entity and Prediction entity is one-to-many.

The Prediction entity is an important component of the system. It stores the output generated by the model. The attributes associated with it are Prediction_ID, Result and Probability Score. Each prediction is linked to a specific input data instance and is generated by using any particular model. This establishes relationships between Input Data, Model and Prediction entities.

An unique feature of this project is the Explanation entity which is related to Explainable AI or XAI. This entity stores all the explanation of all of the predictions that are made. The attributes included are Explanation_ID, Feature_Contribution, SHAP_Values aur Visualization_Type. The relation between production and expendable entity is one to one.

The relationship between these entities ensures smooth data flow within the system. User provides input data so that the model can process it and generate a prediction and all of the prediction are backed by explanation with the use of XAI. The data set supports the model by providing training and testing data.

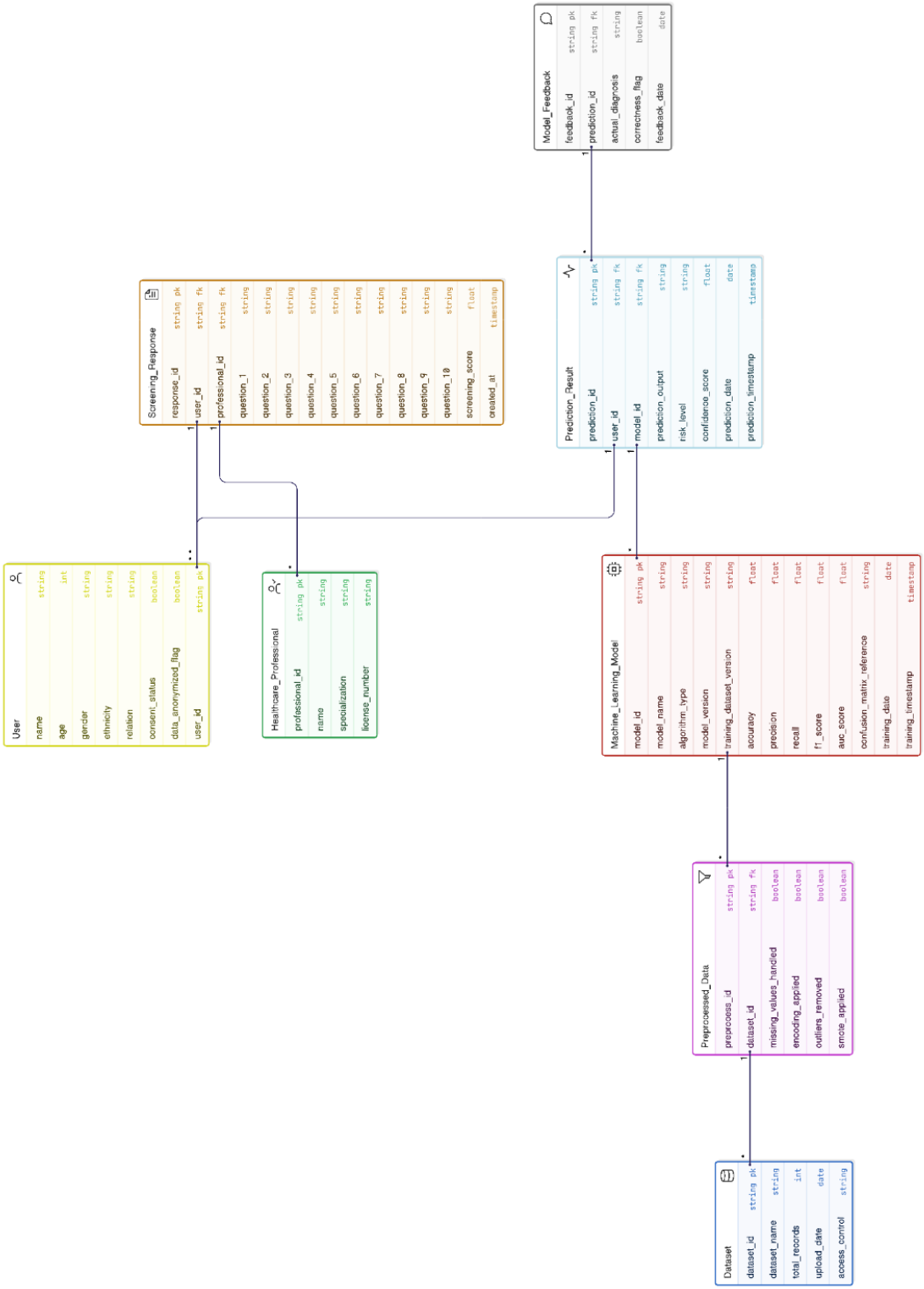


Fig No.3.8.4: ER Diagram

3.8.5 Class Diagram

Class diagram is a fundamental component of object-oriented system design which represents the static structure of the system. It illustrates the relationship between classes, their attributes and methods. For this project, class diagram provides clear understanding of different components of a system, how they are organized and how they interact with each other in an object-oriented manner.

The system is designed with a modular approach where each major functionality of the system is represented in the form of a separate class. The primary classes of the system include User, DataHandler, Preprocessing, ModelTrainer, Predictor, SHAPExplainer and ResultDisplay. Each class has their own specific attributes and methods which define the class's behavior and responsibility.

The user class represents the entity which interacts with the system. It consists of attributes such as user_id, name and role. The primary method of the class is inputData() which provides the facility of providing the user's behavioral and demographic data. This class sends input data to the processing modules of the system and receive the prediction results.

The DataHandler class is responsible for managing the dataset. It includes attributes namely, dataset_path and data_frame. The methods present in this class include loadData() to load the dataset from a CSV file, displayInfo() which shows the details of the dataset and splitData() to divide the dataset into training and testing sets. This class acts as a bridge between raw data and data preprocessing.

The data preprocessing class Handles data cleaning and transformation related tasks The attributes present here are raw_data and processed_data. The methods in this class are handleMissingValues(), encodeCategoricalData(), detectOutliers() and applySMOTE(). This class ensures that the data is present

in a correct format for the machine learning models.

The ModelTrainer class is responsible for training all the machine learning models. It includes attributes such as model_list and trained_model. The methods included here are trainModels(), evaluateModels() and selectBestModel().

This class implements algorithms such as Logistic Regression, Decision Tree, Random Forest, Support Vector Machine, and XGBoost. It also performs cross-validation and hyperparameter tuning.

The Predictor class is used to generate predictions based on new input data. It contains attributes such as input_data and prediction_result. Here, the primary method is predict() which uses the trained model for classifying the input data and determine if the individual is likely to have Autism Spectrum Disorder. This class interacts closely with the ModelTrainer class to use the selected model.

A key component of the system is the SHAP Explainer class. It provides explainability to the predictions. This class contains attributes such as shap_values and feature_importance. The methods include calculateSHAPValues(), generateSummaryPlot() and generateForcePlot().

The ResultDisplay class is responsible for presenting the output to the user. It includes attributes like prediction and explanation. The methods include displayPrediction() and displayExplanation() which is responsible to show the output or result.

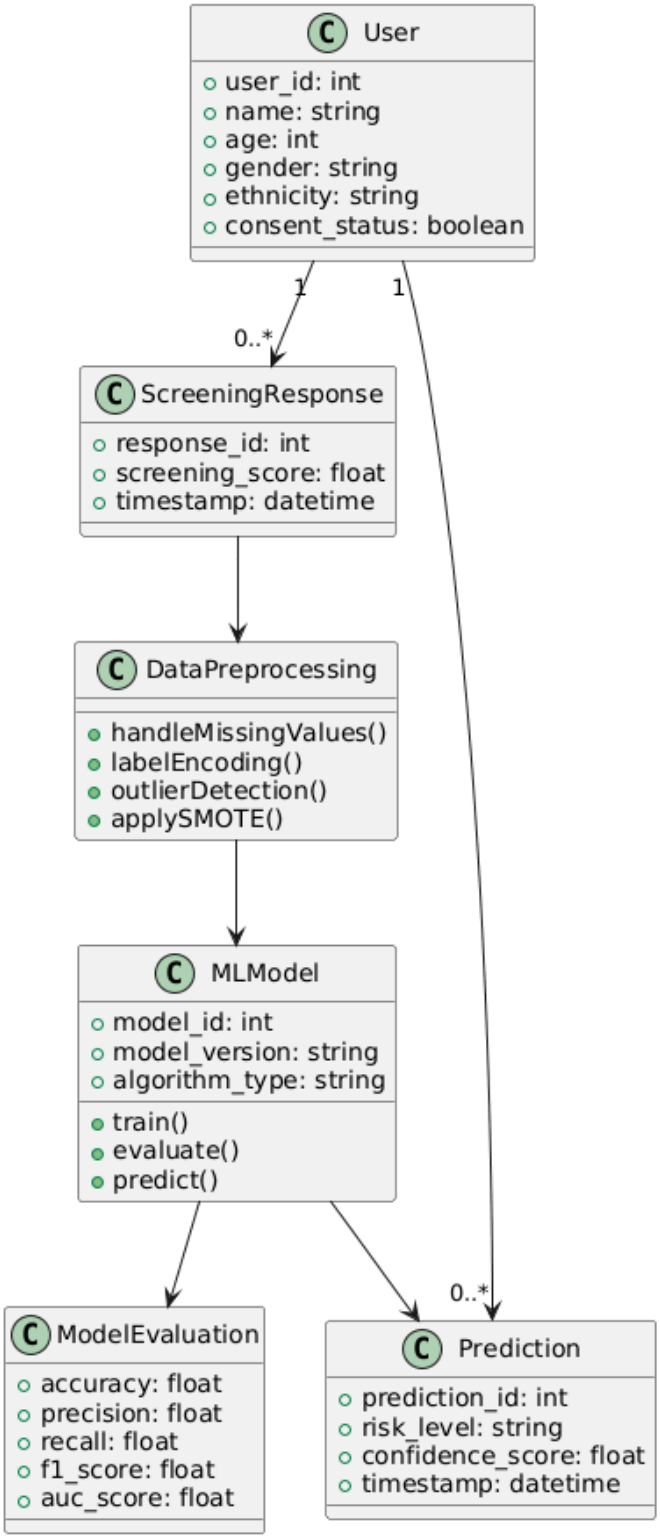


Fig 3.8.5: Class Diagram

3.8.6 0, 1 and 2 level DFDs

With the use of Data Flow Diagrams (DFD), a clear understanding is gained for the system analysis that has been done which is, how the data flows in the system and how different processes interact with each other. For the proposed project, DFD is used to represent the data flow from input to output across various system components. The system has been analyzed through three levels of DFDs: Level 0 (Context Diagram), Level 1 aur Level 2.

DFD Level 0 (Context Diagram)

Level 0 DFD represents the system as a single process and shows it's the interaction with external entities. The primary external entity is User who provides behavioral and demographic details as inputs. The system takes these inputs and processes to generate an output which are prediction result and explanations. Level 0 DFD gives a high-level overview of the system without getting into internal details.

DFD Level 1

The Level 1 DFD breaks the main system in smaller parts and shows how the information flows between these parts. The key processes are as follows:

- Data Input Module – Receives user input or loads the dataset.
- Data Preprocessing Module – Cleans the data, handles missing values, encodes categorical variables, and balances the dataset using SMOTE.
- Exploratory Data Analysis (EDA) Module – It analyzes data patterns and generates visualizations.
- Model Training and Evaluation Module – Machine learning models are trained and evaluated.
- Prediction Module – Input data is used as a base for generating predictions.
- Explainability Module – Adds explainability to the results generated.

DFD Level 2

The Level 2 DFD provides a deeper insight than Level1 DFD. It focuses on individual processes and shows the internal operations. For example, the Data Preprocessing Module can be further divided into:

- Missing value handling
- Categorical data encoding
- Outlier detection
- Data normalization
- SMOTE application for class balancing

Similarly, the Model Training Module includes:

- Train-test splitting
- Model selection
- Cross-validation
- Hyperparameter tuning

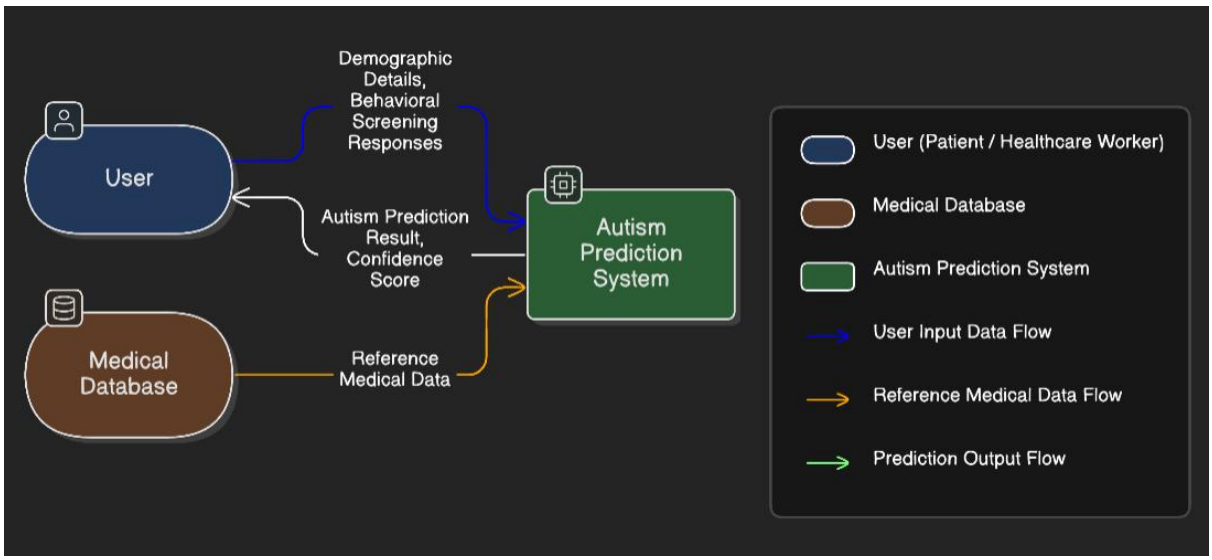


Fig No.3.8.6.1 O Level DFD

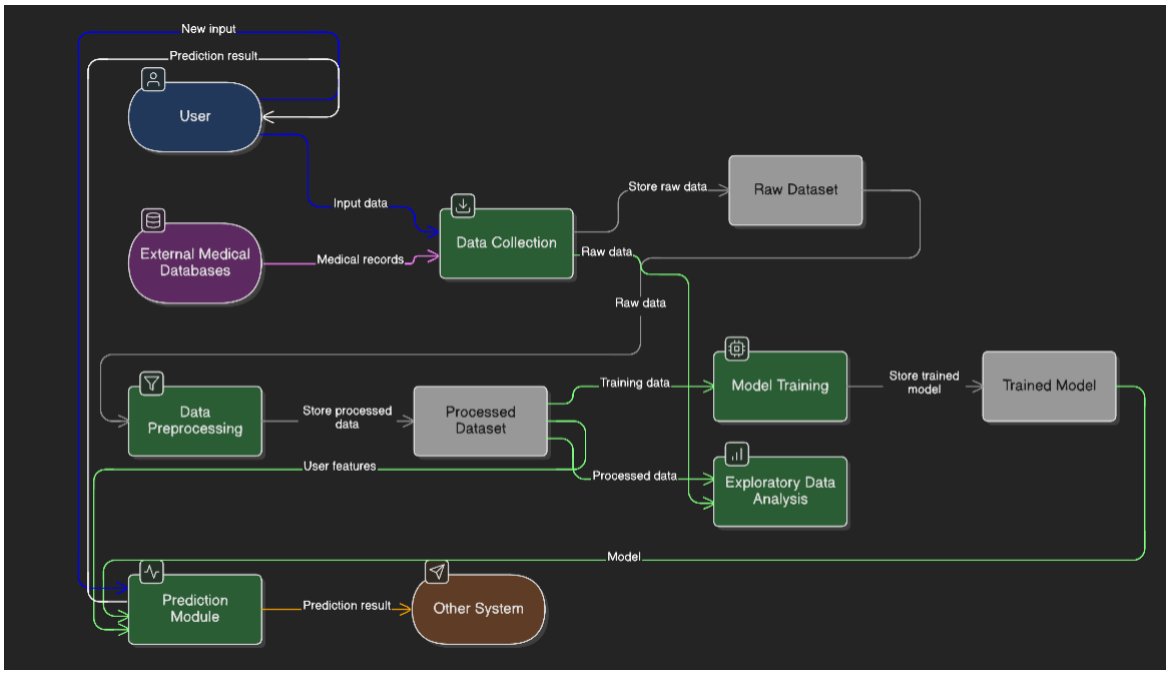


Fig No.3.8.6.2: 1 Level DFD

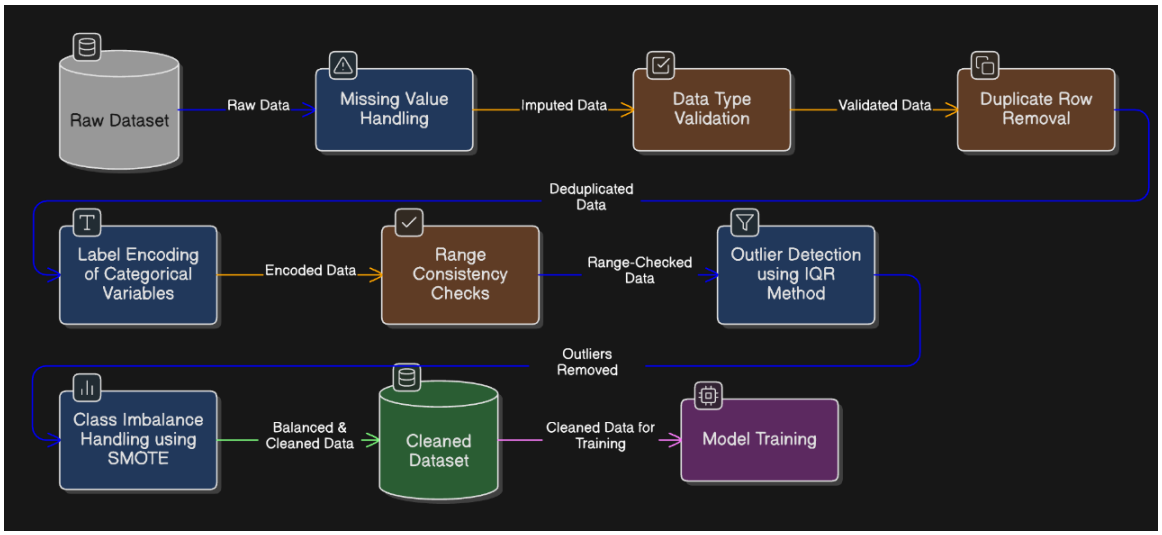


Fig No.3.8.6.3: 2 Level DFD

4. SYSTEM DESIGN

4.1 MODULES DISCRIPTION

Module Identification, Description and Effort Estimation

Proposed project "Autism Spectrum Disorder Prediction using Machine Learning Techniques and Explainable AI (SHAP)" is developed in a structured and modular manner as a part of industrial training project. Development of this project is divided into two main phases: Learning Phase and Developmental Phase.

Phase I – Learning Phase (Curriculum-Based Training)

During the initial phase of the internship theoretical and practical knowledge acquisition was focused upon Which was based on machine learning and python. According to the shared syllabus in this phase Python fundamentals, data structures, file handling, object-oriented programming, data analytics using NumPy aur Pandas, visualization using Matplotlib and Seaborn, statistical foundations, supervised learning techniques, cross-validation, ensemble learning methods and evaluation metrics and such topics are covered following a module wise approach.

Estimated Effort: 30 person-days

Phase II – Project Implementation Modules

The project is divided into six major modules as follows:

Module 1: Data Collection Module

Effort: 5 person-days

For carrying out the project, open source, public data repositories such as Kaggle are taken in account for the dataset that is primarily being used. Although a questionnaire-based form was developed and circulated by me in order to understand the real-world data acquisition and building a dataset. This module is also concerned with validation checks which ensures the input data

is complete and correct.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 800 entries, 0 to 799
Data columns (total 22 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   ID               800 non-null    int64
 1   A1_Score         800 non-null    int64
 2   A2_Score         800 non-null    int64
 3   A3_Score         800 non-null    int64
 4   A4_Score         800 non-null    int64
 5   A5_Score         800 non-null    int64
 6   A6_Score         800 non-null    int64
 7   A7_Score         800 non-null    int64
 8   A8_Score         800 non-null    int64
 9   A9_Score         800 non-null    int64
 10  A10_Score        800 non-null    int64
 11  age              800 non-null    float64
 12  gender           800 non-null    object
 13  ethnicity        800 non-null    object
 14  jaundice         800 non-null    object
 15  austim           800 non-null    object
 16  contry_of_res    800 non-null    object
 17  used_app_before  800 non-null    object
 18  result           800 non-null    float64
 19  age_desc         800 non-null    object
 20  relation         800 non-null    object
 21  Class/ASD        800 non-null    int64
dtypes: float64(2), int64(12), object(8)
memory usage: 137.6+ KB
```

Fig No.4.4.1.1: Figure shows the attributes for the dataset used along with the data type

Module 2: Data Preprocessing Module

Effort: 8 person-days

This is a core technical module. It is concerned with handling missing values, categorical encoding, outlier detection and using IQR method and class

imbalance handling. The clean data is forwarded for model training or prediction stages.

The data set consists of categorical features such as gender, ethnicity, country of residence, and other attributes which cannot be directly used with, machine learning models. Therefore, Label Encoding is applied so that the categorical values can be converted into numerical format.

Label encoding assigns a unique integer to each category which is easily processed by the model. This transformation ensures that all of the features are in suitable format for machine learning algorithms to work upon.

```
encoders
```

```
{'gender': LabelEncoder(),
 'ethnicity': LabelEncoder(),
 'jaundice': LabelEncoder(),
 'austim': LabelEncoder(),
 'contry_of_res': LabelEncoder(),
 'used_app_before': LabelEncoder(),
 'relation': LabelEncoder()}
```

Fig No.4.4.1.2: Figure shows the label encoders

Outlier Detection and Treatment using IQR Method

Outliers are those extreme values which deviate the data majority significantly and have negative impact on the model's performance. In this project, outliers are detected through Interquartile Range (IQR) method.

The IQR is calculated as the difference between the third quartile (Q3) and the first quartile (Q1). Any value falling below Q1-1.5 x IQR or above Q3 + 1.5 x IQR is considered as an outlier. Instead of removing these values, they are replaced with the median of respective feature. Median is chosen because it is less sensitive to extreme values and helps in maintaining the stability.

Handling Class Imbalance using SMOTE

In real world datasets, class imbalance is a common issue where the samples of one class are way more than the samples of another class. An initial imbalance has been noticed in the classes that represent autism spectrum disorder in the data set that has been used.

To address this issue, SMOTE (Synthetic Minority Oversampling Technique) is applied to the training data. SMOTE generates synthetic samples for the minority classes by creating new data points based on existing ones.

After applying SMOTE, both the classes are balanced equally. As shown in the output below, each class has the same number of samples. This improves the model's ability to learn from both classes effectively and reduces bias.

SMOTE (Synthetic Minority Oversampling technique)

```
smote = SMOTE(random_state=42)
```

```
X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)
```

```
print(y_train_smote.shape)
```

```
(1030,)
```

```
print(y_train_smote.value_counts())
```

```
1    515
0    515
Name: Class/ASD, dtype: int64
```

Fig No.4.4.1.3: SMOTE balances the classes

Module 3: Exploratory Data Analysis (EDA) Module

Effort: 6 person-days

This module is responsible to analyze the statistical properties of the dataset. It generates summary statistics, correlation heat maps, graphs, boxplots and so on which help in identifying patterns and dependencies.

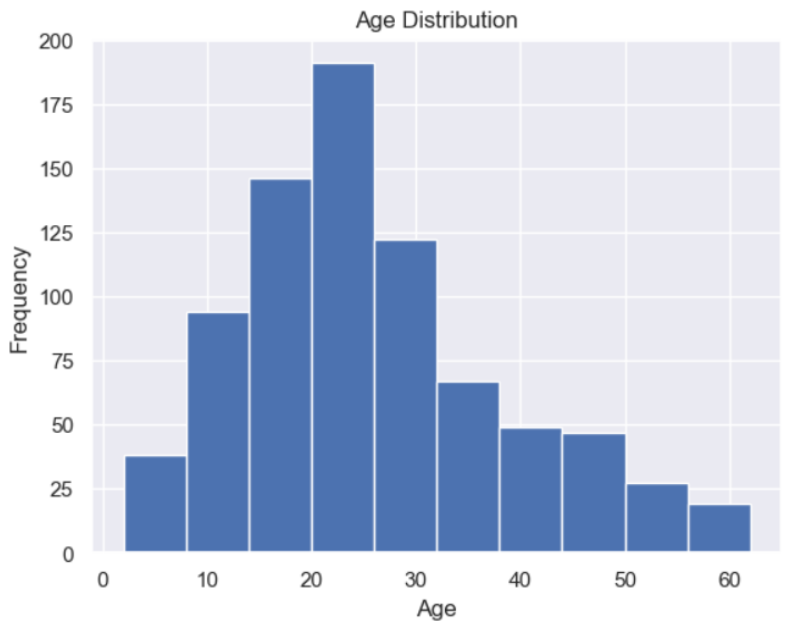


Fig No.4.4.1.4: Age Distribution of Individuals

The age distribution in Fig No.4.4.1.4 shows that the majority of individuals fall within the age group of approximately 15 to 30 years. This indicates a concentration of younger participants in the dataset. The distribution appears to be slightly right-skewed, where fewer individuals are present in higher age groups (above 40 years). This suggests that the dataset mostly represents younger populations, which may influence the model's learning behavior.

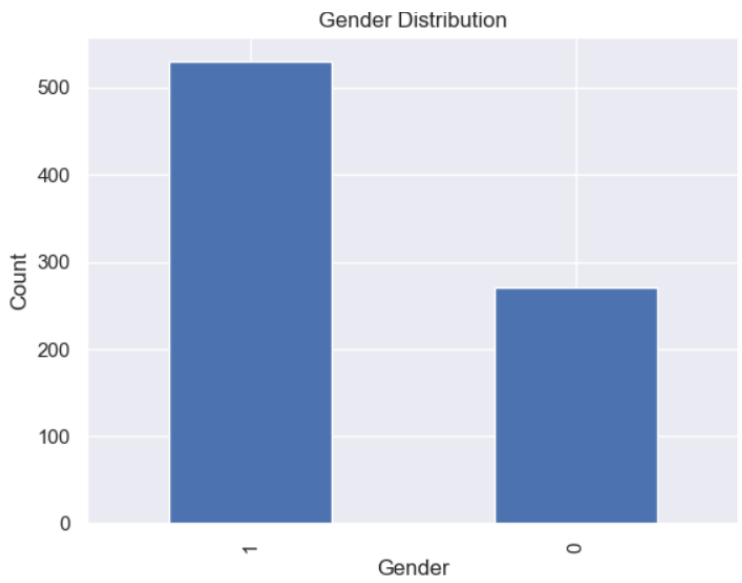


Fig No.4.4.1.5: Gender Distribution

Gender distribution indicates that there is an imbalance between both the categories. The data set consist of '1' (male) labelled individuals more in number than the individuals labelled with '0' (female). This imbalance can introduce biasness in the model if not handled properly. Hence, during model development, resampling or waiting are few techniques which can be considered.

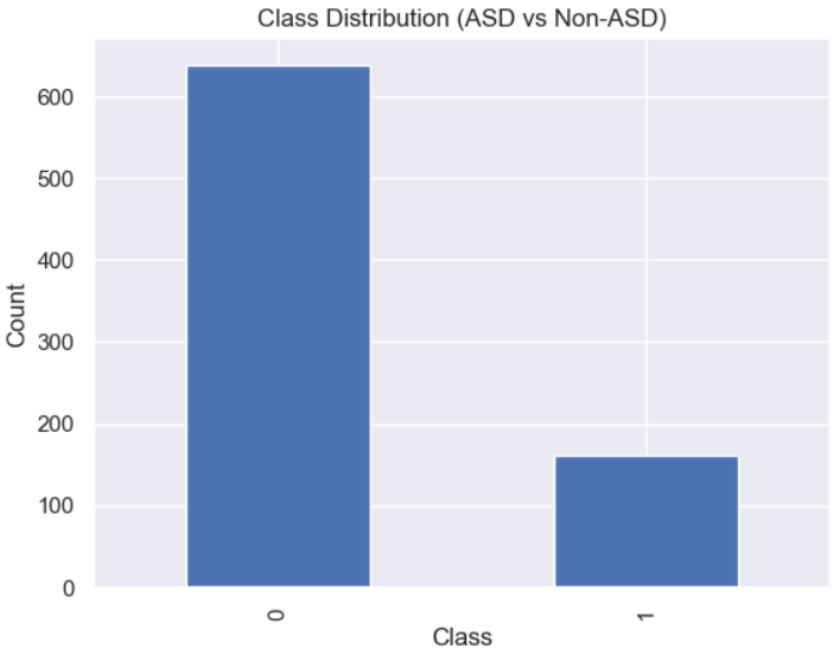


Fig No.4.4.1.6: Class Distribution (ASD vs Non-ASD)

Class distribution clearly shows that there is an imbalance between both the classes. Non-ASD (0) cases re way too many than ASD (1) cases. This imbalance is critical because it makes the model bias towards the majority class .Hence, SMOTE, under sampling or class weighting are few techniques which can be considered in order to ensure better model performance and fairness in results.

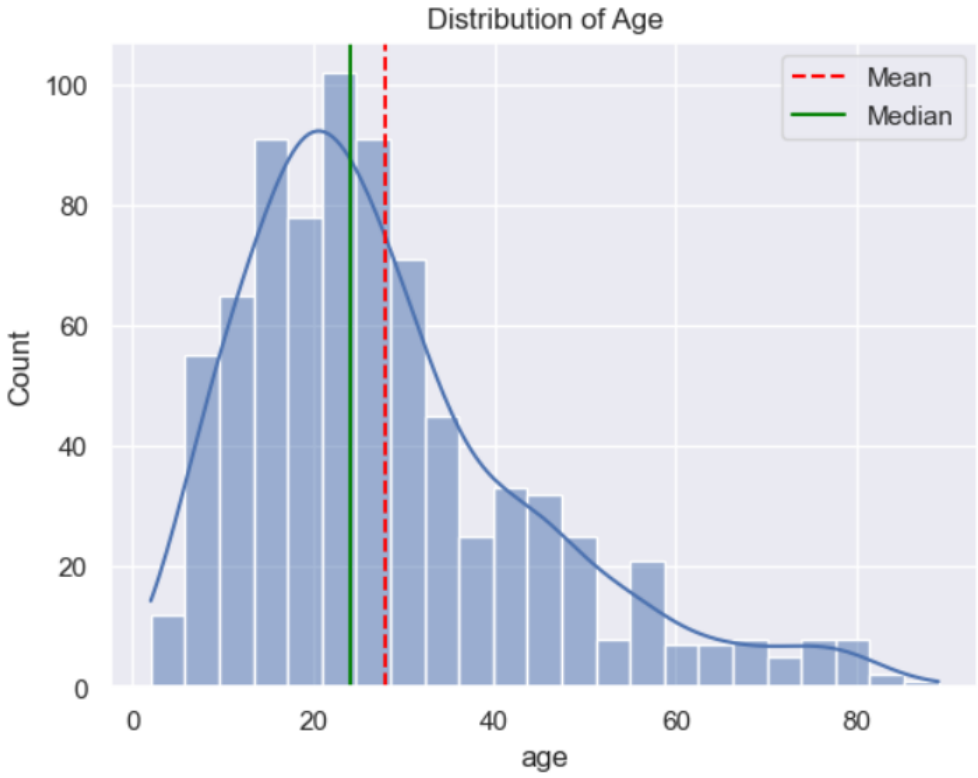


Fig No.4.4.1.7: Distribution of Age with Mean and Median

The Fig No.4.4.1.7 represents the distribution of the age feature in the dataset using a histogram. The plot provides insights into how age values are spread across the dataset. The red dashed line indicates the mean age, wheareas the green dashed line represents the median age. The mean is 27.96375 and median is 24.

This analysis helps in understanding the central tendency and distribution pattern of the data. It also helps in identifying any skewness or irregularities in the dataset. Such insights are important for selecting appropriate preprocessing techniques for improving the performance of the model.

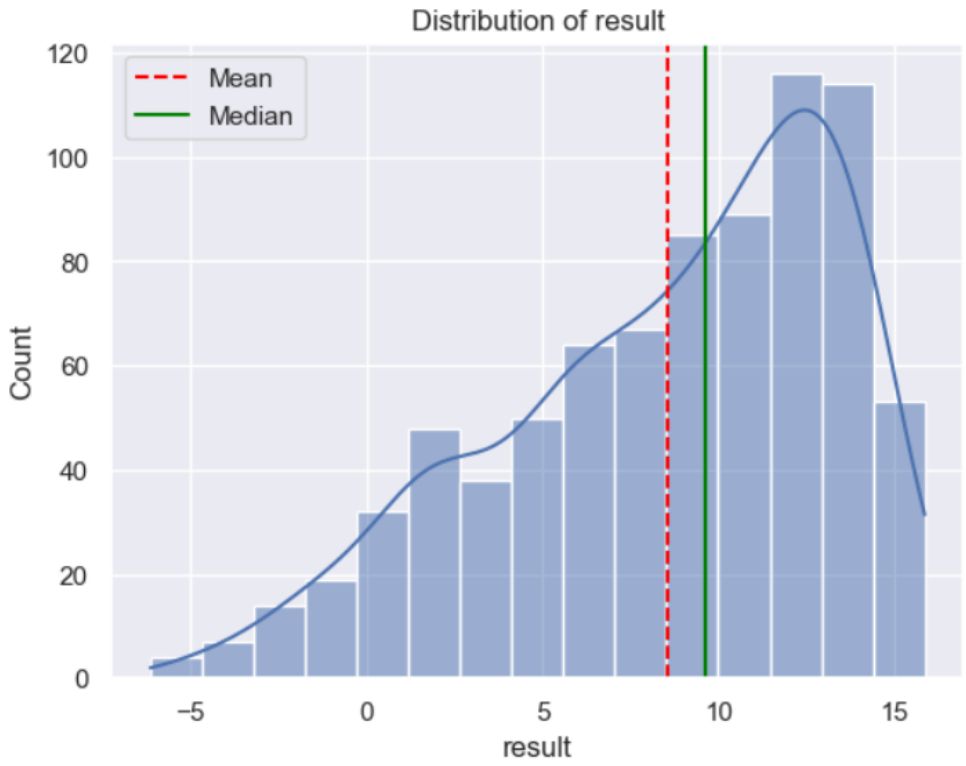


Fig No.4.4.1.8: Distribution of Screening Result Score with Mean and Median

The Fig No.4.4.1.8 shows the distribution of the result feature. It represents the screening score obtained from behavioral assessment questions. The histogram shows how the result values are distributed across the dataset.

Red dash line indicates the mean score whereas, green line indicates median score. This analysis helps to understand the overall scoring pattern and to identify if the data is skewed towards higher or lower values.

Because the result score is directly related to autism screening, it is very important to analyze its distribution so that the structure of the dataset can be understood and check how it is affecting the model predictions.

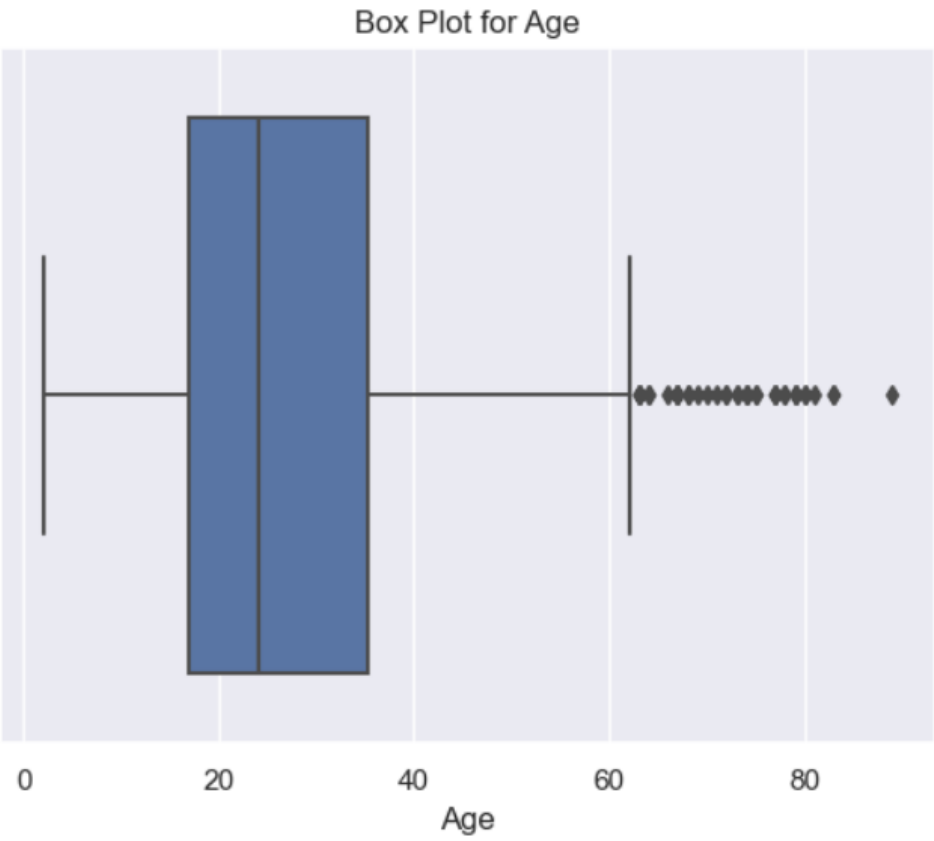


Fig No.4.4.1.9: Box Plot for Age Showing Distribution and Outliers

The Fig No.4.4.1.9 represents a box plot of the age feature. It is used to analyze the distribution and identify outliers in the dataset. The box indicates that the interquartile range (IQR) contains the middle 50% of the data and the line inside the box represents the median age.

The whiskers extend to the minimum and maximum values in an acceptable range and the points outside of the whiskers represent the outliers. Points with higher age values are observed as outliers in this dataset.

Outlier detection is important because extreme values affect the model's performance. Identifying these values helps in deciding the preprocessing techniques whether removal or transformation is required.

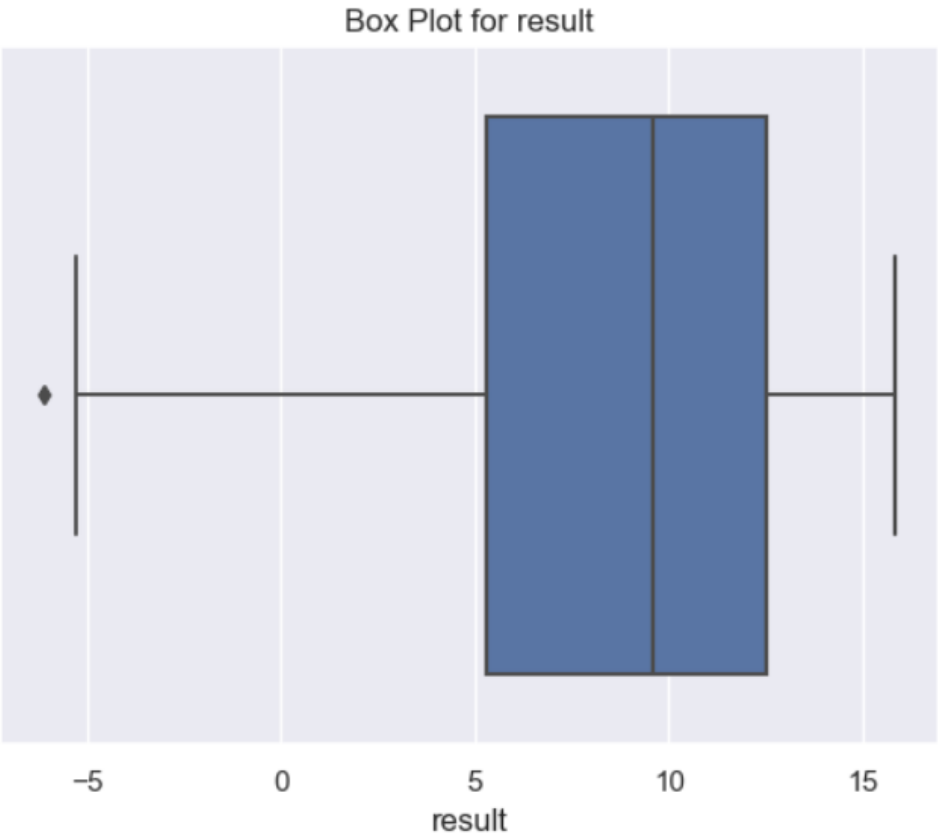


Fig No.4.4.1.10: Box Plot for Result Showing Distribution and Outliers

The box plot of result feature illustrates the distribution of screening scores and help in identifying the outliers. It represents plot scores with median values.

If outliers are present, then they are represented with unusually high or low screening scores. The analysis of these values helps in ensuring better data quality, also improving the reliability of machine learning models.

Univariate Analysis of Categorical Features

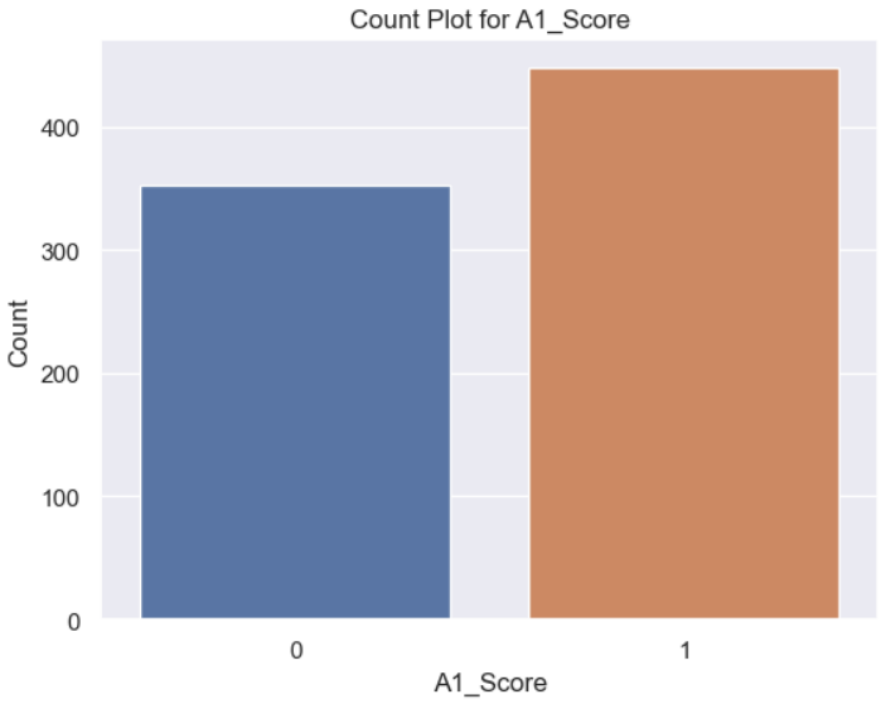


Fig No.4.4.1.11: Count Plot for A1_Score

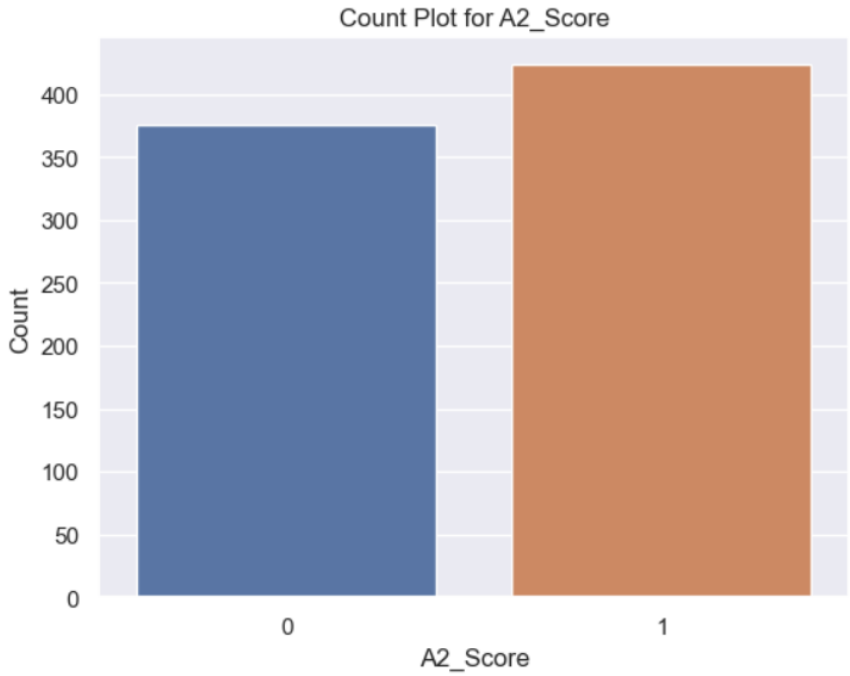


Fig No.4.4.1.12: Count Plot for A2_Score

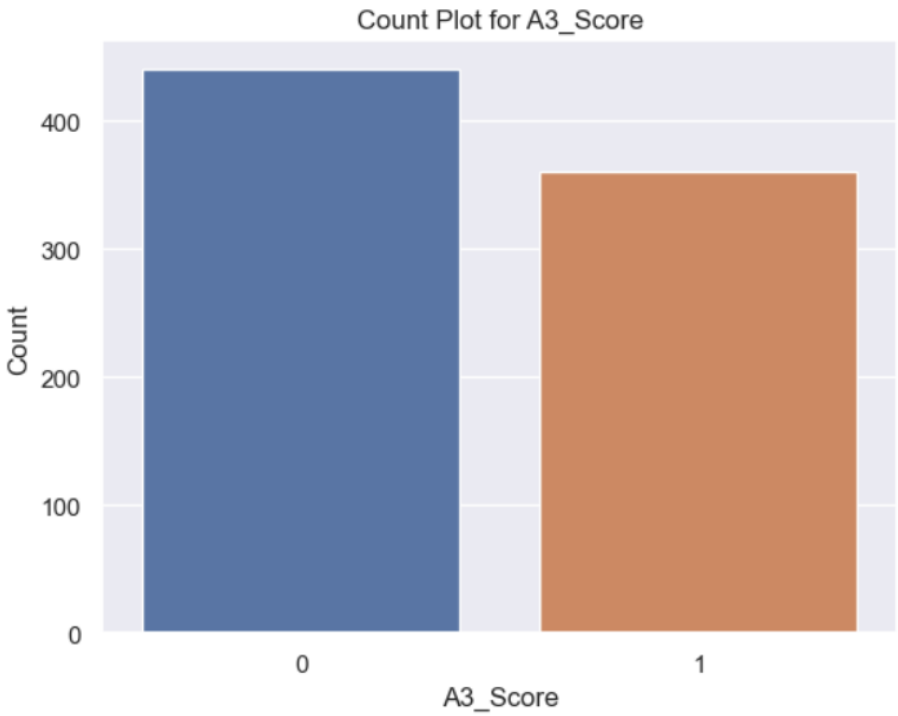


Fig No.4.4.1.13: Count Plot for A3_Score

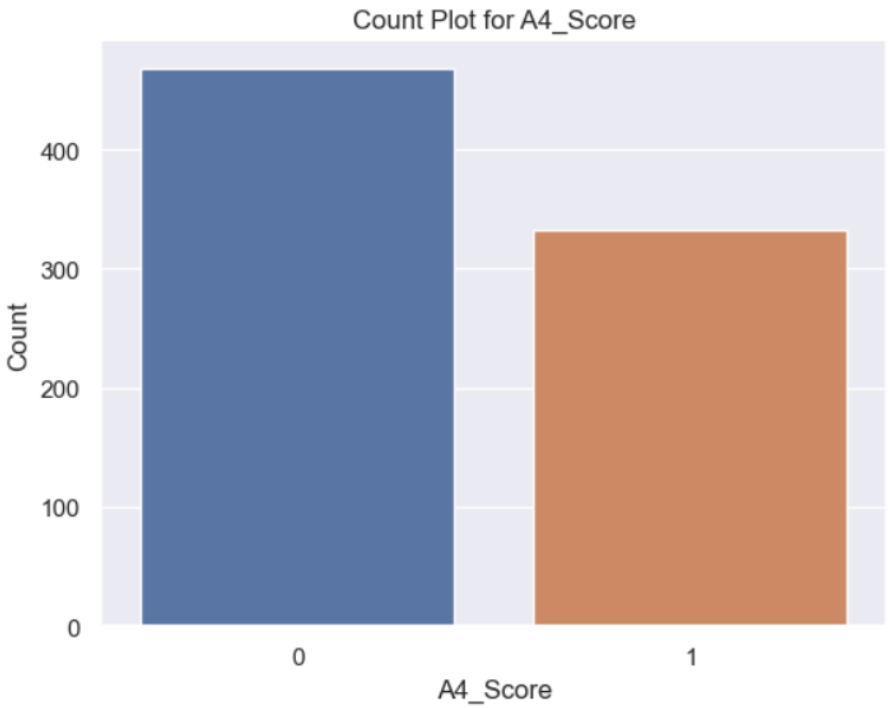


Fig No.4.4.1.14: Count Plot for A4_Score

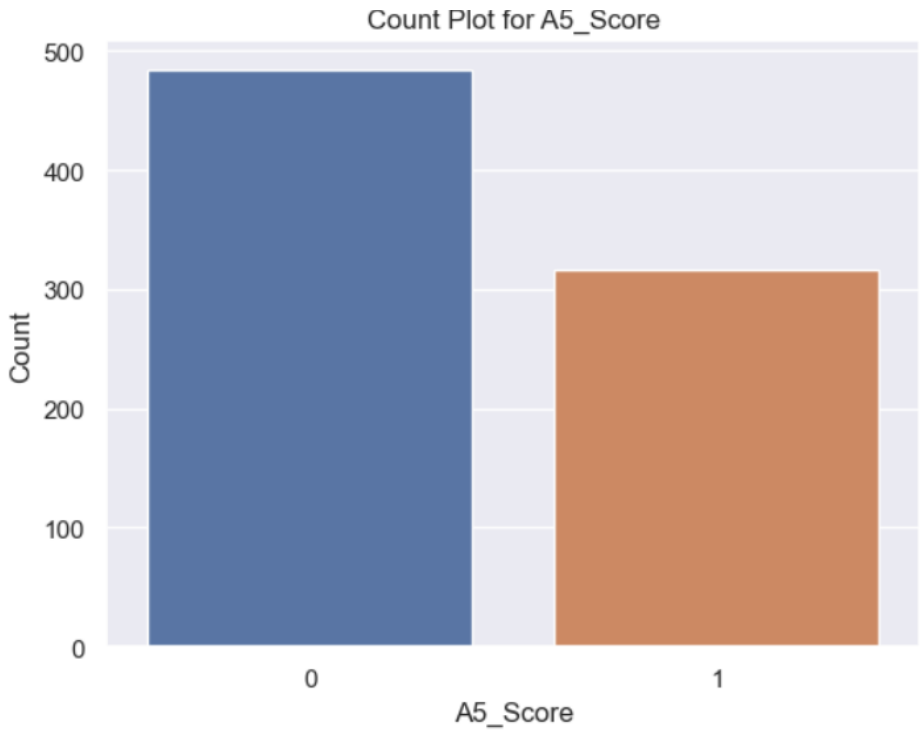


Fig No.4.4.1.15: Count Plot for A5_Score

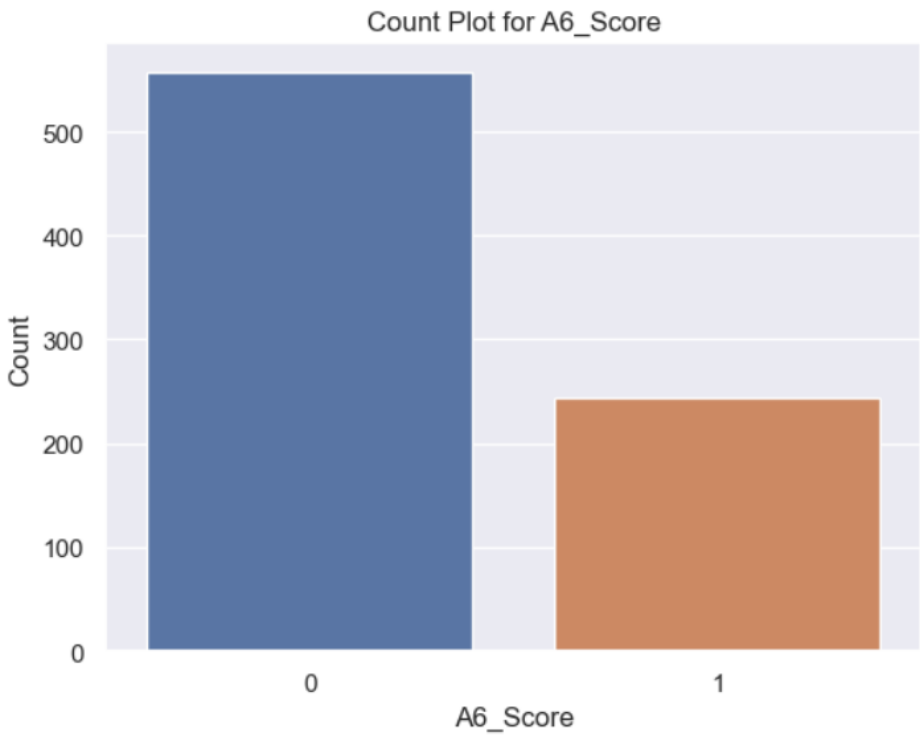


Fig No.4.4.1.16: Count Plot for A6_Score

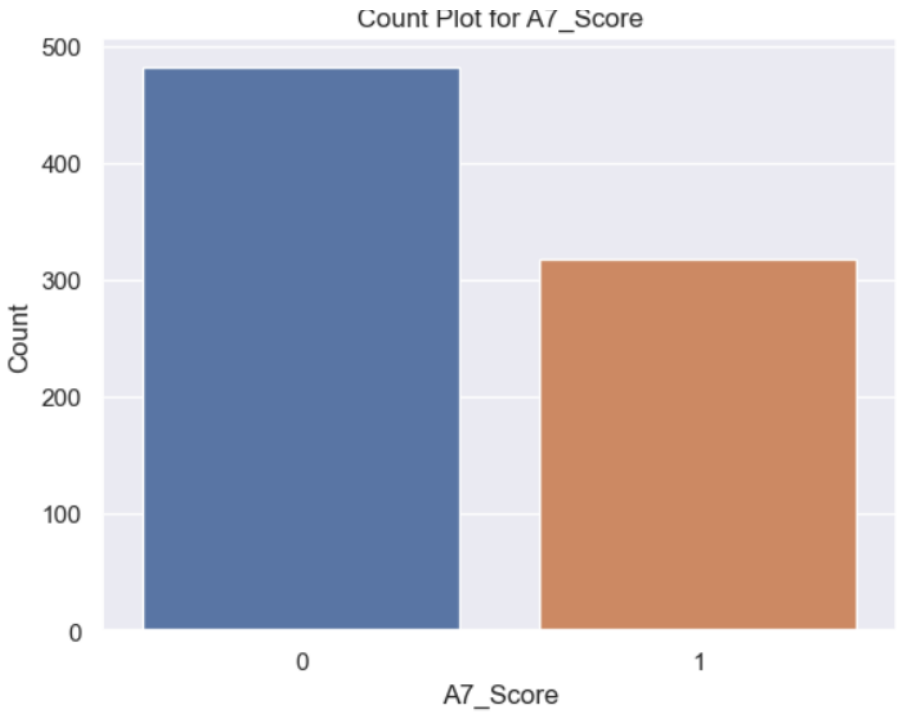


Fig No.4.4.1.17: Count Plot for A7_Score

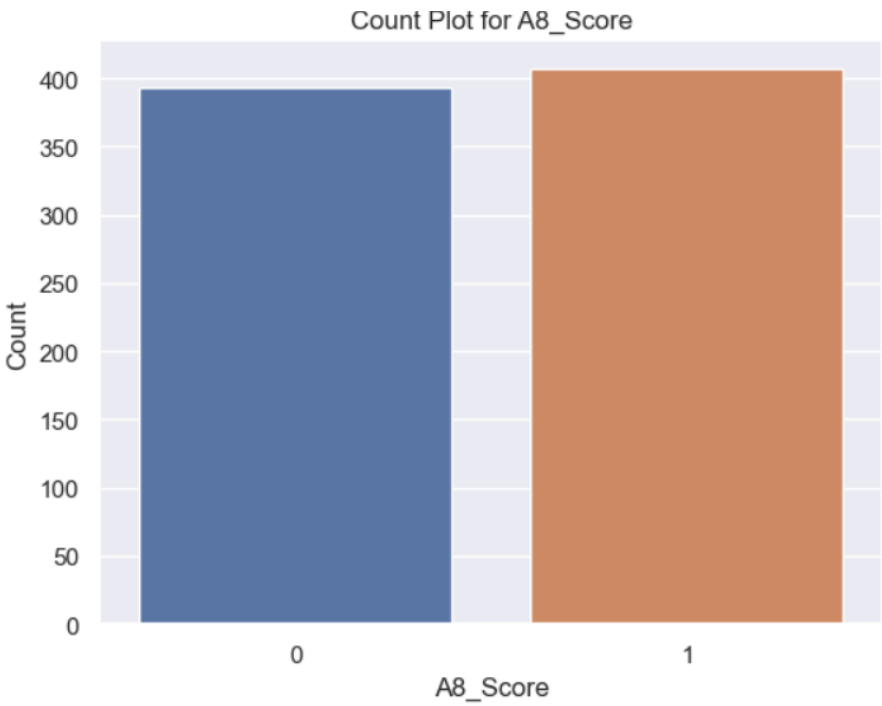


Fig No.4.4.1.18: Count Plot for A8_Score

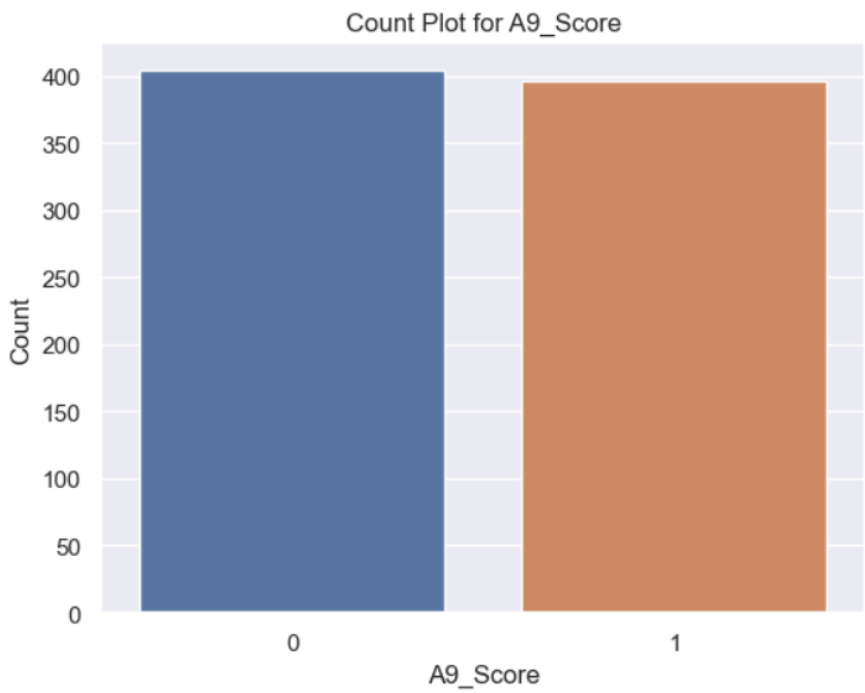


Fig No.4.4.1.19: Count Plot for A9_Score

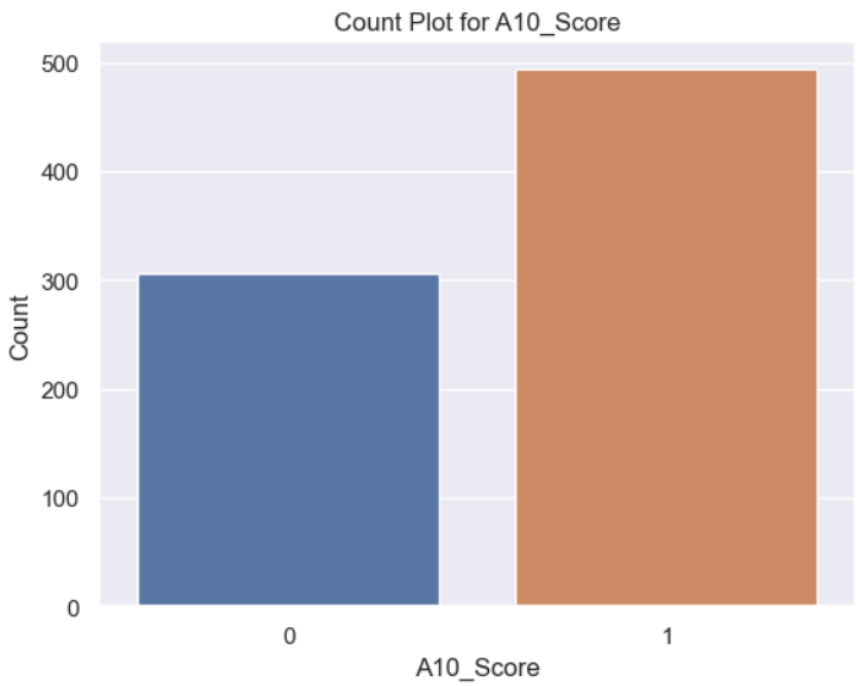


Fig No.4.4.1.20: Count Plot for A10_Score

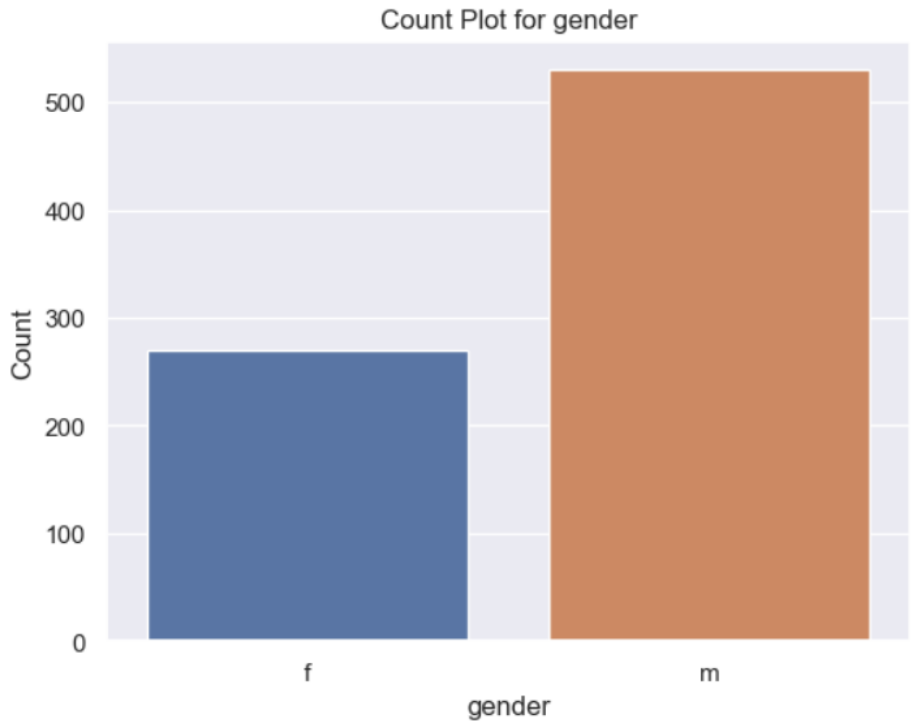


Fig No.4.4.1.21: Count Plot for gender

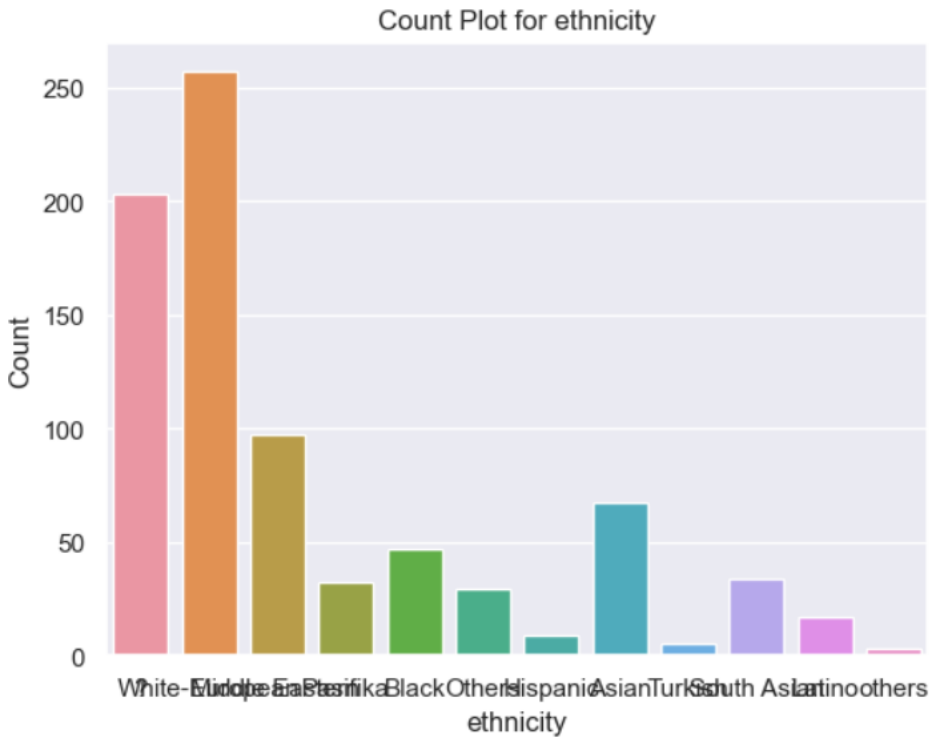


Fig No.4.4.1.22: Count Plot for ethnicity

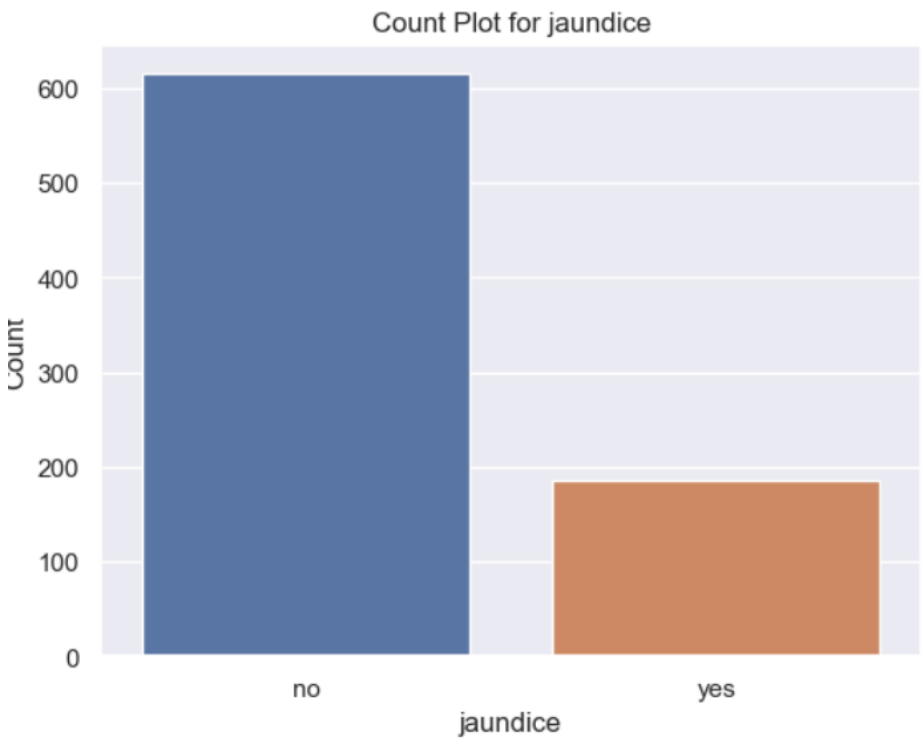


Fig No.4.4.1.23: Count Plot for jaundice

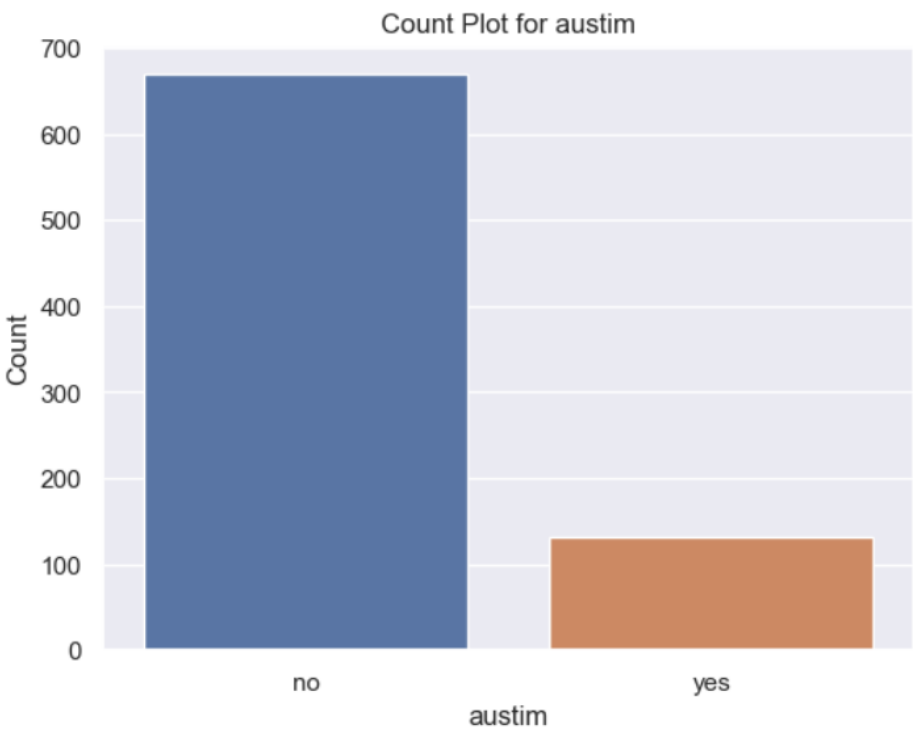


Fig No.4.4.1.24: Count Plot for autism

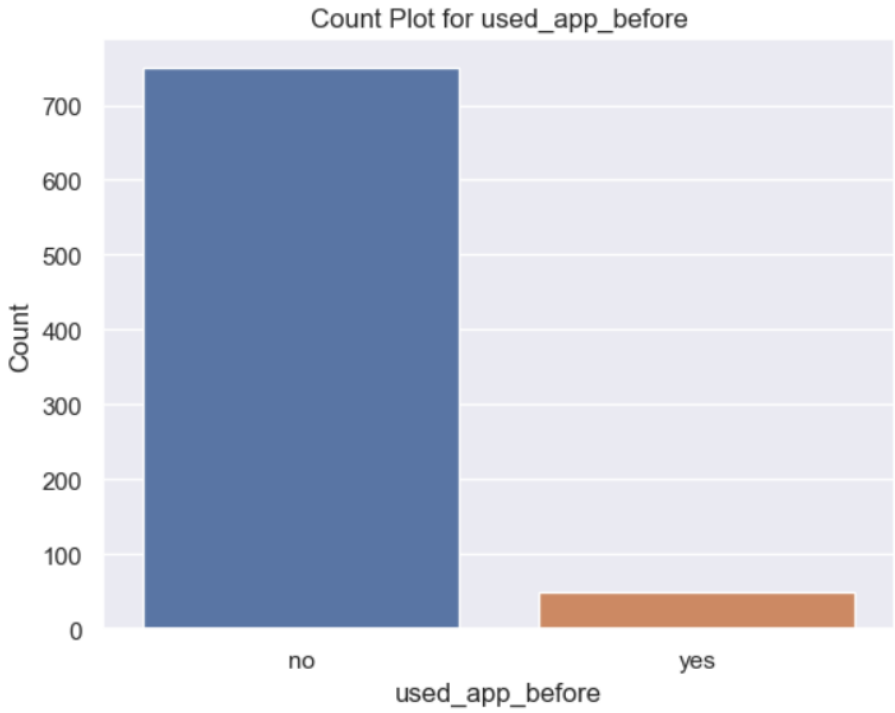


Fig No.4.4.1.25: Count Plot for used_app_before

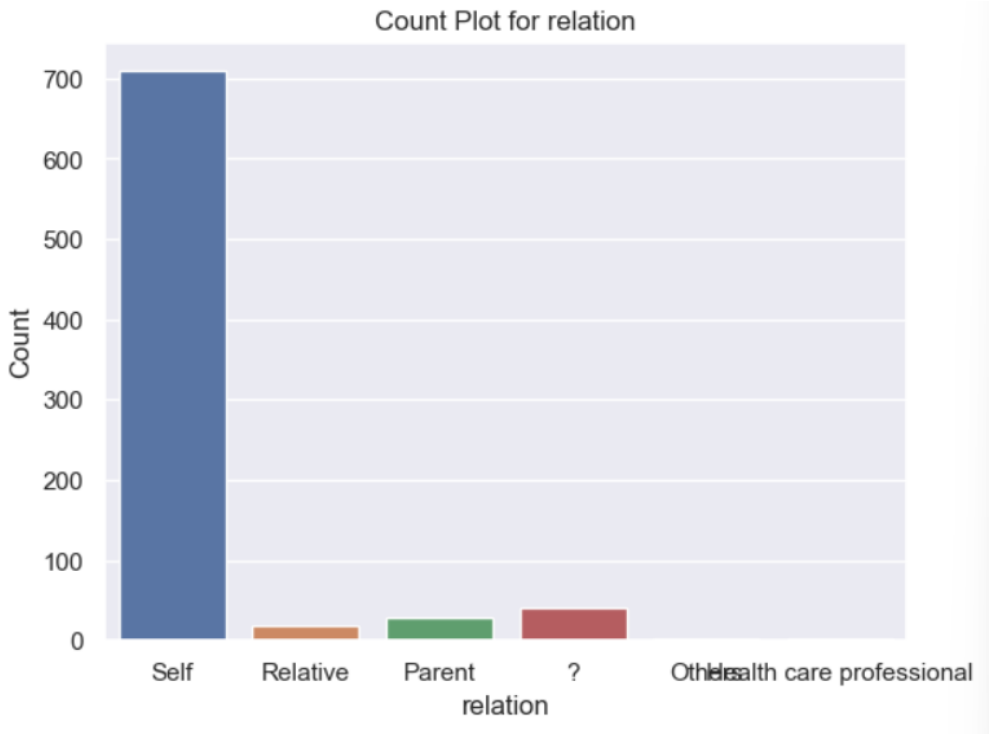


Fig No.4.4.1.26: Count Plot for relaation

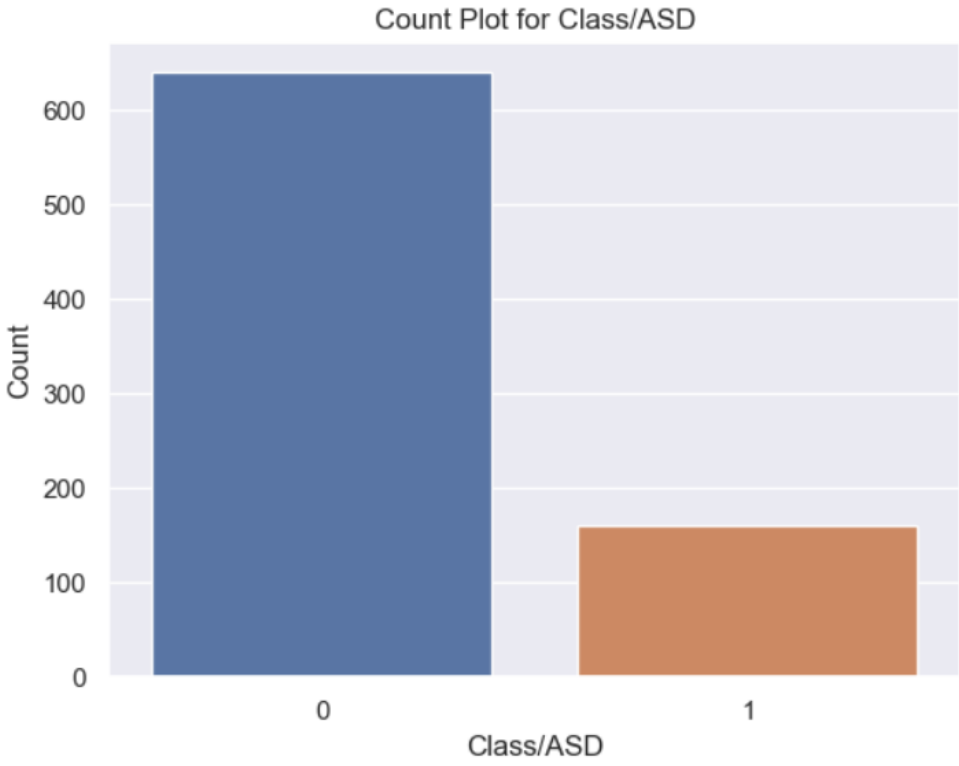


Fig No.4.4.1.27: Count Plot for Class/ASD

The figures (Fig No.4.4.1.11 - Fig No.4.4.1.27) represent the count plots of categorical features such as A1_Score to A10_Score, gender, ethnicity, jaundice, autism and other related attributes. These plots show the frequency distribution of each category within the dataset.

For the screening score features (A1 to A10), the values typically represent binary responses (0 for No and 1 for Yes). The count plots helps us in understanding how responses are distributed among individuals for each question.

Similarly, categorical variables such as gender, ethnicity, and jaundice provide insights into demographic and medical attributes of the dataset. Analyzing these distributions is helpful in identifying any imbalance in the data. It is also important for building accurate machine learning models.

This analysis also assists in understanding the entire composition of the dataset. It ensures that appropriate preprocessing techniques are applied wherever it is necessary.

Bivariate Analysis using Correlation Heatmap

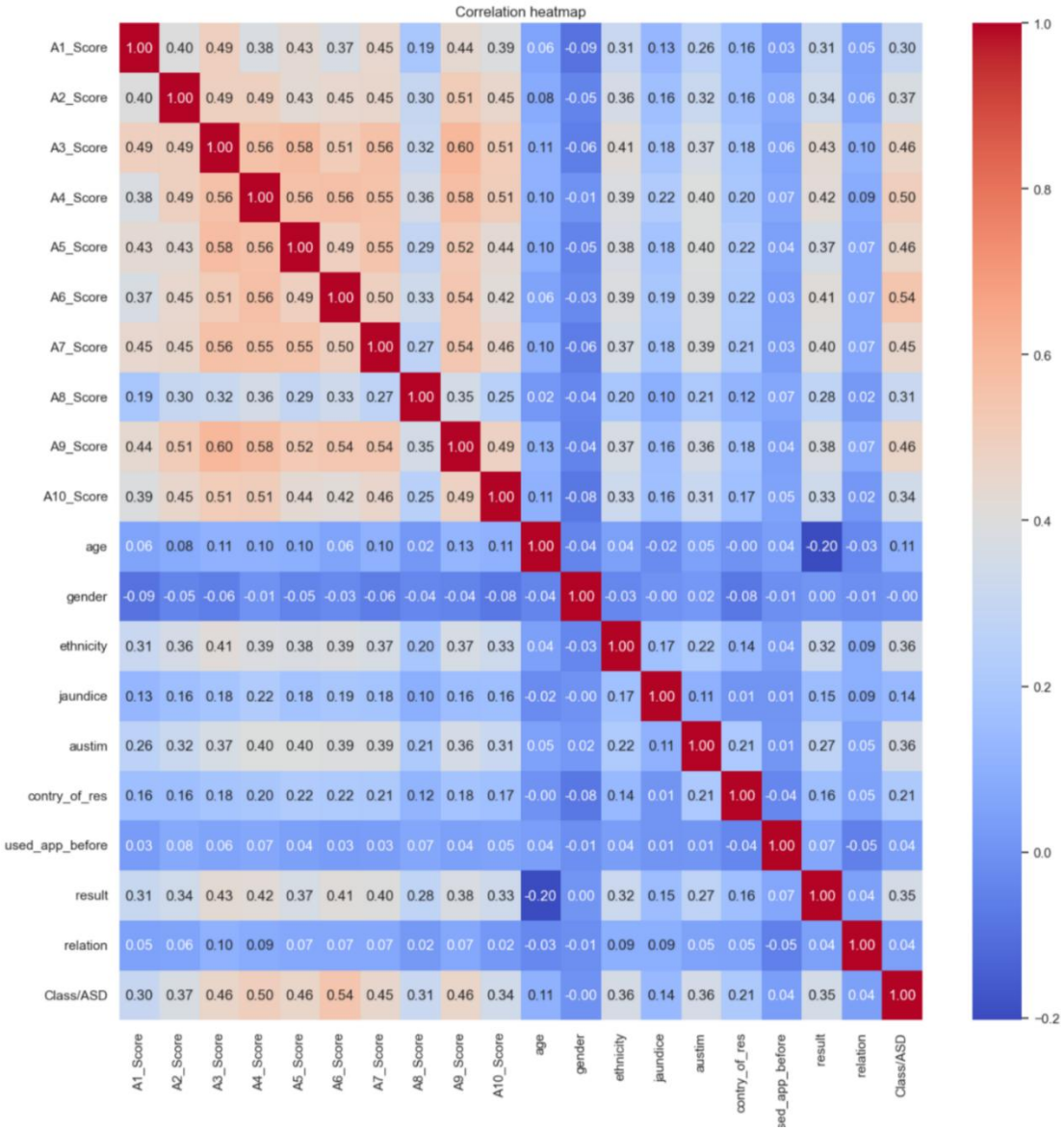


Fig No.4.4.1.28: Correlation Heatmap Showing Relationships Between Features

The Fig No.4.4.1.28 represents the correlation heatmap of the dataset used. It illustrates the relationship between different features. The correlation values range from -1 to +1, where values closer to +1 indicate a strong positive

correlation, values closer to -1 indicate a strong negative correlation and values near 0 indicate weak or no correlation.

From the heatmap, it can be observed that several screening score features (A1_Score to A10_Score) show moderate positive correlation with each other, indicating that these behavioral attributes are interrelated. Additionally, some features exhibit a noticeable correlation with the result feature, suggesting their importance in predicting Autism in individuals.

The heatmap helps in identifying important features and understanding dependencies within the dataset. It also assists in detecting redundant features that may not contribute significantly to the model, thereby improving feature selection and overall model performance.

- Most A_Score features show moderate positive correlation (around 0.4–0.6).
- No strong negative correlation is observed among major features.
- The result feature shows moderate correlation with multiple screening scores, indicating their predictive importance.

Module 4: Model Training and Evaluation Module

Effort: 10 person-days

This module will implement supervised classification algorithms such as Logistic Regression, Decision Tree, Random Forest, and Support Vector Machine. It will perform train-test split, cross-validation, hyperparameter tuning, and model comparison to select the best-performing model.

Data Splitting into Training and Testing Sets

Before training the machine learning models, the dataset is divided into input features and target variable. The feature set (X) contains all independent variables such as screening scores, age, and demographic attributes, while the target variable (y) represents the class label indicating the presence or absence of Autism.

This separation is essential as it allows the model to learn patterns from the input data and predict the target variable. The dataset is then split into training and testing sets, where the training data is used to train the model and the testing data is used to evaluate its performance on unseen data. This approach ensures that the model generalizes well and helps in avoiding overfitting.

The dataset is further divided into training and testing sets using the train_test_split function, typically in an 80:20 ratio. This ensures that the model is trained on a majority of the data while reserving a portion for performance evaluation.

```
     A1_Score  A2_Score  A3_Score  A4_Score  A5_Score  A6_Score  A7_Score  \
0           1         0         1         0         1         0         1
1           0         0         0         0         0         0         0
2           1         1         1         1         1         1         1
3           0         0         0         0         0         0         0
4           0         0         0         0         0         0         0
..        ...       ...       ...       ...       ...       ...       ...
795         0         1         0         0         0         0         0
796         0         1         1         0         0         1         0
797         0         0         0         0         0         0         0
798         0         0         0         0         0         0         0
799         0         1         0         0         0         0         0

     A8_Score  A9_Score  A10_Score   age  gender  ethnicity  jaundice  austim  \
0           0         1          1  38.0       0          5         0       0
1           0         0          0  47.0       1          5         0       0
2           1         1          1   7.0       1          9         0       1
3           0         0          0  23.0       0          5         0       0
4           0         0          0  43.0       1          5         0       0
..        ...       ...        ...   ...     ...        ...       ...     ...
795         0         1          1  16.0       1          2         0       0
796         1         1          1  20.0       1          9         0       0
797         0         0          0   5.0       1          7         1       0
798         0         0          0  16.0       0          5         0       0
799         0         0          0  46.0       0          5         0       0

     contry_of_res  used_app_before     result  relation
0                6                0   6.351166         1
1               23                0   2.255185         1
2               52                0  14.851484         1
3               52                0   2.276617         1
4               44                0  -4.777286         1
..             ...              ...        ...       ...
795             34                0  12.999501         1
796             16                0  13.561518         1
```

Fig No.4.4.1.29: Print(x) show the train attributes

```
0      0
1      0
2      1
3      0
4      0
      ..
795    0
796    0
797    0
798    0
799    0
```

Fig No.4.4.1.30: Print(y) show the test attributes

Model Training and Cross-Validation

```
Training Decision Tree with default parameters...
Decision Tree Cross-Validation Accuracy: 0.86
--------------------------------------------------
Training Random Forest with default parameters...
Random Forest Cross-Validation Accuracy: 0.92
--------------------------------------------------
Training XGBoost with default parameters...
XGBoost Cross-Validation Accuracy: 0.90
--------------------------------------------------
```

Fig No.4.4.1.30: Training and Evaluation of Multiple Machine Learning Models

In this stage, multiple machine learning algorithms are implemented to predict Autism Spectrum Disorder. The models used in this project include Decision Tree, Random Forest, and XGBoost.

Each model is trained using the preprocessed and balanced dataset obtained after applying SMOTE. To ensure reliable performance, 5-fold cross-validation is used. In this technique, the dataset is divided into five subsets,

and the model is trained and validated multiple times to obtain an average accuracy score.

The results show that Random Forest achieved the highest cross-validation accuracy of 0.92, followed by XGBoost and Decision Tree. Based on performance comparison, the best model is selected for final prediction.

Cross-validation helps in improving model reliability and reduces the chances of overfitting.

Hyperparameter Tuning for Model Optimization

To improve the performance of machine learning models, hyperparameter tuning is performed. Hyperparameters are configuration settings that control the behavior of the learning algorithm and significantly impact model accuracy.

In this project, different parameter combinations are defined for Decision Tree, Random Forest, and XGBoost models. These parameters include max_depth, number of estimators, learning rate, and minimum samples split.

A search technique such as RandomizedSearchCV or GridSearchCV is used to evaluate multiple parameter combinations and identify the optimal configuration for each model. This process helps in enhancing model performance and ensures better generalization on unseen data.

Hyperparameter tuning plays a crucial role in improving prediction accuracy and building a more robust and efficient machine learning model.

```python
# fit the models
random_search_dt.fit(X_train_smote, y_train_smote)
random_search_rf.fit(X_train_smote, y_train_smote)
random_search_xgb.fit(X_train_smote, y_train_smote)
```

```
RandomizedSearchCV
estimator: XGBClassifier
XGBClassifier
XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=None, device=None, early_stopping_rounds=None,
              enable_categorical=False, eval_metric=None, feature_types=None,
              gamma=None, grow_policy=None, importance_type=None,
              interaction_constraints=None, learning_rate=None, max_bin=None,
              max_cat_threshold=None, max_cat_to_onehot=None,
              max_delta_step=None, max_depth=None, max_leaves=None,
              min_child_weight=None, missing=nan, monotone_constraints=None,
              multi_strategy=None, n_estimators=None, n_jobs=None,
              num_parallel_tree=None, random_state=42, ...)
```

Fig No.4.4.1.31: RandomizedSearchCV Applied on XGBoost Model

In this project, RandomizedSearchCV is used with the XGBoost classifier to identify the best set of parameters. The model is trained on the preprocessed and balanced dataset, and multiple iterations are performed to find the optimal configuration.

The output shows the trained model with the selected parameters after the tuning process. This approach improves model performance and ensures better generalization on unseen data.

Final Model Selection and Model Saving

```
# Get the model with best score

best_model = None
best_score = 0

if random_search_dt.best_score_ > best_score:
  best_model = random_search_dt.best_estimator_
  best_score = random_search_dt.best_score_

if random_search_rf.best_score_ > best_score:
  best_model = random_search_rf.best_estimator_
  best_score = random_search_rf.best_score_

if random_search_xgb.best_score_ > best_score:
  best_model = random_search_xgb.best_estimator_
  best_score = random_search_xgb.best_score_
```

```
print(f"Best Model: {best_model}")
print(f"Best Cross-Validation Accuracy: {best_score:.2f}")
```

```
Best Model: RandomForestClassifier(bootstrap=False, max_depth=20, n_estimators=50,
                        random_state=42)
Best Cross-Validation Accuracy: 0.93
```

```
# save the best model
with open("best_model.pkl", "wb") as f:
  pickle.dump(best_model, f)
```

Fig No.4.4.1.32: Selection of Best Performing Model Based on Cross-Validation

After performing hyperparameter tuning on multiple machine learning models, the best-performing model is selected based on cross-validation accuracy. The models evaluated include Decision Tree, Random Forest, and XGBoost.

The model with the highest cross-validation score is chosen as the final model. In this case, the Random Forest classifier achieved the highest accuracy of 0.93, making it the most suitable model for prediction.

Once the best model is selected, it is saved using the pickle library. This allows the trained model to be reused for future predictions without retraining, improving efficiency and enabling deployment in real-world applications.

Module 5: Prediction Module

Effort: 6 person-days

This module will load the trained model and generate predictions for new user inputs. It will compute probability scores and classify risk levels (High/Low).

Reporting and Documentation Module

Effort: 6 person-days

This module will generate performance reports, confusion matrix outputs, cross-validation summaries, and store prediction logs.

Data Structures and Process Logic

The system will utilize appropriate data structures and logical flow mechanisms aligned with the Machine Learning curriculum.
Data Structures used are as follows:

1. **Pandas DataFrame** Used for dataset storage and manipulation. It handles rows (records) and columns (features).
2. **NumPy Arrays:** Used for numerical computations and model input matrices.
3. **Lists:** Used to store feature names and model comparison results.
4. **Dictionaries:** Used for label encoding mappings and parameter storage.
5. **Train-Test Split Structure:** Divides dataset into training and testing sets.
6. **Pickle Files:** Used to serialize and store trained models and encoders.

Process Logic of Each Module

User Input Logic

- Accept demographic details.
- Validate required fields.
- If invalid → show error.
- If valid → send to preprocessing.

Preprocessing Logic

- Check for missing values.
- Replace missing entries.
- Encode categorical variables.
- Detect and treat outliers.
- Apply SMOTE (during training).

EDA Logic

- Calculate mean, median, standard deviation.
- Generate visualizations.
- Analyze feature correlations.

Model Training Logic

- Split dataset (70:30 or 80:20).
- Train multiple algorithms.
- Perform cross-validation.
- Tune hyperparameters.
- Compare evaluation metrics.
- Select best model.
- Save model.

Prediction Logic

- Load saved model.
- Apply preprocessing to user input.
- Run prediction function.

- Compute probability.
- Classify risk level.
- Display output.

Reporting Logic

- Generate confusion matrix.
- Compute accuracy, precision, recall, F1-score.
- Save prediction results.
- Prepare summary report.

Reports Likely to be Generated

- Model Performance Report
- Cross-Validation Report
- Hyperparameter Tuning Summary
- Confusion Matrix Report
- Prediction Summary Report
- Dataset Preprocessing Report
- EDA Visualization Report

Proposed Security Mechanisms at Various Levels

Since the proposed system deals with healthcare-related data, including demographic information and behavioral screening responses, implementing appropriate security mechanisms is essential. Although the project is academic in nature, the design will incorporate multiple layers of security to ensure confidentiality, integrity, and reliability of data.

Data-Level Security

At the data level, personal information such as age, gender, and screening responses will be handled securely. The following mechanisms are proposed:

1. Data anonymization: Personally identifiable information will not be permanently stored in raw format during model training.
2. Validation checks: Input validation will prevent injection of invalid or malicious data.
3. Controlled dataset access: Training datasets imported from external sources will be stored locally with restricted access.
4. Secure file handling: Dataset files and trained model files (pickle format) will be stored in secure directories with limited permissions.

Application-Level Security

At the application layer, the following security mechanisms are proposed:

1. Input validation and sanitization to prevent runtime errors or unexpected behavior.
2. Exception handling to manage system crashes and avoid exposure of system internals.
3. Controlled access to training functions to prevent unauthorized retraining of the model.
4. Model version control to ensure traceability and prevent accidental overwriting.

Model-Level Security

Machine learning models can be vulnerable to misuse or unintended modification. Therefore, the following safeguards are proposed:

1. Serialized model protection using secure pickle storage.
2. Restricted retraining permissions.

3. Validation of prediction input to avoid adversarial or abnormal data entries.

4. Performance monitoring to detect model degradation.

System-Level Security

At the system level, standard software security practices will be followed:

1. Use of updated Python libraries.

2. Proper error logging without revealing sensitive information.

3. Backup of datasets and trained models.

4. Secure handling of temporary files during preprocessing.

Ethical and Privacy Considerations

Since the system relates to mental health awareness:

1. User consent must be obtained before data submission.

2. The system will clearly state that predictions are supportive tools and not medical diagnoses.

3. No sensitive medical diagnosis data will be permanently stored without authorization.

Explainable AI (SHAP)

The SHAP module works by analyzing both global and local explanations. Global explanations provide an overview of feature importance across the entire dataset, while local explanations focus on individual predictions. This helps in understanding which factors increase or decrease the likelihood of Autism for a specific case. By integrating SHAP, the system overcomes the "black-box" nature of traditional machine learning models.

4.2 USER INTERFACE DESIGN

The user interface of the proposed system is designed to be simple, intuitive, and user-friendly, ensuring ease of interaction for both technical and non-technical users. Although the current implementation is performed in a Jupyter Lab environment, a conceptual interface is proposed for real-world deployment.

The interface will consist of an input section where users can enter behavioral and demographic details such as age, gender, and screening scores. The input fields will be designed using dropdowns and text boxes to minimize errors and improve usability.

Once the data is submitted, the system will process the input and display the prediction result in a clear format, indicating whether the individual is at risk of Autism. Additionally, the interface will include a dedicated section for explainability, where SHAP-based visualizations such as feature importance graphs and contribution plots will be displayed.

The interface will also provide proper instructions and validation messages to guide users during data entry. Error handling mechanisms will ensure that incorrect or incomplete inputs are flagged before processing.

For future enhancement, the system can be developed as a web-based application using frameworks such as Flask or Streamlit, making it accessible to a wider audience.

5. TESTING

5.1 TESTING TECHNIQUES AND TESTING REPORTS

Testing is a crucial phase for the development of any software system. It ensures that the system is correct, efficient and functions in a reliable way. Testing in the proposed project has been used for testing the accuracy of predictions, robustness of the model and validate the overall system performance.

In the testing process, trained machine learning models are evaluated on unseen data using a test dataset. It helps to determine how much of the model is generalized outside of the training data. To assess the effectiveness of the model, performance metrics such as accuracy, precision, recall and F1 scores are used. These metrics provide comprehensive understanding of how much the system is accurate in classifying individuals with autism.

As a part of the testing process, confusion matrix is also used which helps to analyze model's prediction in detail. It provides insight about true positives, true negatives, false positive and false negatives. It helps in identifying the areas where the model can make incorrect predictions, especially in minority classes.

In addition to performance evaluation, the system is tested for data handling and preprocessing accuracy. It also includes verification of the missing values, outliers and categorical variables ensuring they are handled properly. It also validates that all the data preprocessing techniques are functioning in alignment of the objective of the project.

Testing Objectives:

1. Functionality: The primary objective of functionality testing is to ensure that all components of the system operate as intended. This includes verifying that each module such as data preprocessing, model training, prediction, and explainability performs its designated tasks correctly. The system is tested to confirm that it produces accurate outputs for valid inputs and that all features work without errors or inconsistencies.

2. Command Handling: Command handling testing focuses on ensuring that the system correctly processes user inputs and commands. The objective is to verify that the system responds appropriately to different types of inputs, including valid, invalid, and edge-case scenarios. Proper command handling ensures that the system remains stable and user-friendly, even when unexpected inputs are provided.

3. Response Accuracy: The objective of response accuracy testing is to evaluate how precisely the system predicts Autism Spectrum Disorder. This involves comparing predicted outputs with actual values using performance metrics such as accuracy, precision, recall, and F1-score. The goal is to ensure that the model provides reliable and consistent predictions across different datasets.

4. Error Handling: Error handling testing aims to ensure that the system can gracefully manage unexpected situations such as invalid inputs, missing data, or runtime errors. The objective is to prevent system crashes and provide meaningful error messages to users. This enhances system reliability and ensures smooth operation under various conditions.

5. Performance: The objective of performance testing is to assess the efficiency and speed of the system during data processing and prediction. The system is evaluated to ensure that it delivers results within an acceptable time

frame without excessive computational overhead. Efficient performance is essential for providing a seamless user experience.

6. Scalability: Scalability testing ensures that the system can handle increasing amounts of data and user load. The objective is to verify that the system remains efficient and responsive when working with larger datasets or when deployed in real-world environments with multiple users.

7. Cross-platform Compatibility: The objective of cross-platform compatibility testing is to ensure that the system functions correctly across different environments. This ensures that the system can be used on different devices without compatibility issues.

8. User Experience: User experience testing focuses on ensuring that the system is easy to use, intuitive, and user-friendly. The objective is to provide a smooth interaction flow, clear outputs, and understandable explanations (especially through SHAP visualizations). A good user experience enhances usability and user satisfaction.

9. Integration: The objective of integration testing is to ensure that all modules of the system work together seamlessly as a unified system. This testing verifies that data flows correctly between modules and that there are no compatibility issues. It ensures that the output of one module serves as the correct input for the next, maintaining consistency and accuracy throughout the system.

10. Regression Testing: The objective of regression testing is to ensure that any updates or modifications made to the system do not negatively affect existing functionality. Whenever changes are introduced, such as tuning model parameters or improving preprocessing techniques, regression testing is performed to confirm that previously working features continue to operate correctly.

5.2 Test Reports

After selecting the best-performing model, it is evaluated using the test dataset to measure its performance on unseen data. The evaluation metrics used include accuracy, confusion matrix, and classification report.

The model achieved an accuracy of approximately 0.82, indicating that it correctly predicts the outcome in most cases. The confusion matrix provides a detailed breakdown of correct and incorrect predictions, showing true positives, true negatives, false positives, and false negatives.

The classification report includes precision, recall, and F1-score for each class. Precision indicates how many predicted positive cases are actually correct, while recall shows how well the model identifies actual positive cases. The F1-score provides a balance between precision and recall.

```
Accuracy score:
 0.81875
Confusion Matrix:
 [[108  16]
 [ 13  23]]
Classification Report:
              precision    recall  f1-score   support

           0       0.89      0.87      0.88       124
           1       0.59      0.64      0.61        36

    accuracy                           0.82       160
   macro avg       0.74      0.75      0.75       160
weighted avg       0.82      0.82      0.82       160
```

Fig No. 5.2.1: Evaluation Metrics of the Final Model on Test Data

Model Explainability using SHAP (Explainable AI)

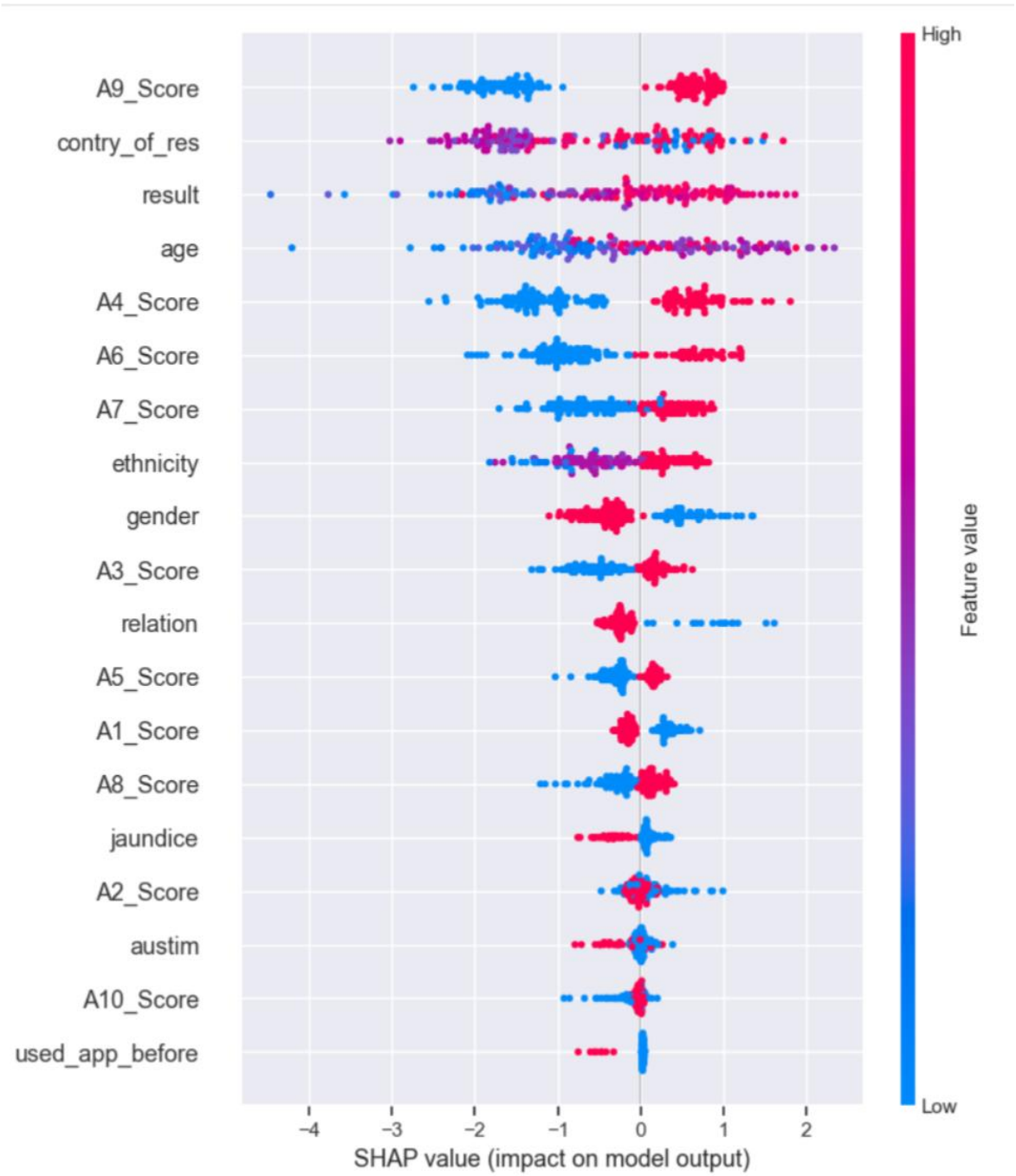


Fig No. 5.2.2: SHAP Summary Plot Showing Feature Impact on Prediction

The SHAP summary plot provides a global explanation of the model by showing the impact of each feature on the prediction. Each point represents a data instance, and the color indicates whether the feature value is high or low.

From the plot, it can be observed that features such as A9_Score, result, and age have a significant influence on the model's decision. Features on the right side contribute positively towards predicting Autism, while those on the left contribute negatively. This visualization helps in understanding which features are most important in the prediction process.

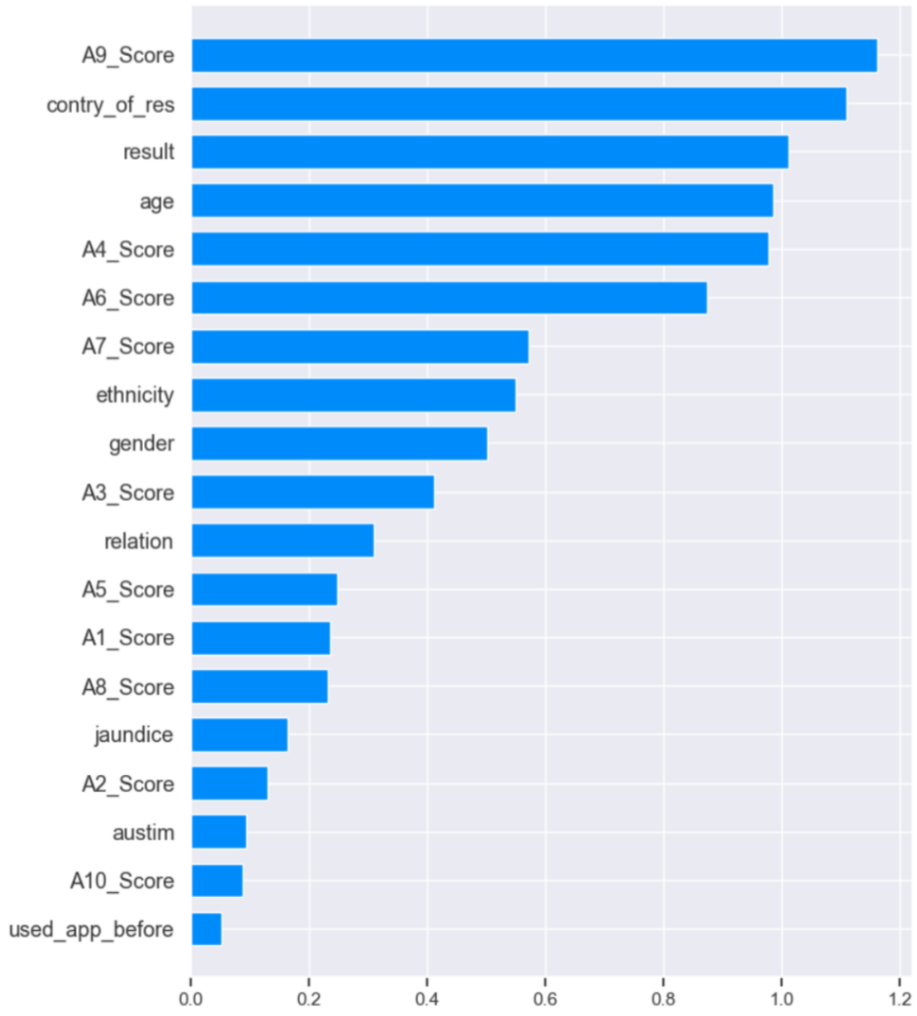


Fig No. 5.2.3: SHAP Feature Importance Bar Plot

The SHAP bar plot represents the global importance of features based on their average contribution to the model output. It ranks features according to their influence on predictions.

From the plot, A9_Score and result are among the most important features, indicating that they play a crucial role in predicting Autism.

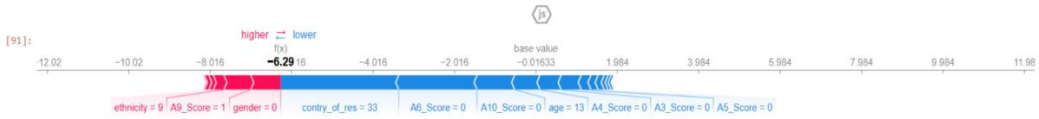


Fig No. 5.2.4: SHAP Force Plot Explaining Individual Prediction

The above figure represents a SHAP force plot, which provides a local explanation for an individual prediction made by the machine learning model. This visualization shows how each feature contributes to pushing the prediction towards a particular class.

In the plot, the base value represents the average prediction of the model. From this base value, different features either increase or decrease the prediction score. Features shown in red push the prediction towards a higher value (indicating a higher likelihood of Autism), while features shown in blue push the prediction towards a lower value (indicating a lower likelihood).

Each feature contributes differently to the final prediction. For example, features such as A9_Score and ethnicity may increase the prediction value, while features like age, A4_Score, and A5_Score may decrease it. The final output value is the result of the combined effect of all these feature contributions.

This visualization helps in understanding the reasoning behind a single prediction and provides transparency by clearly showing how each feature influences the outcome.

6. SYSTEM SECURITY MEASURES

In Security is a critical aspect of any system, especially when dealing with sensitive data such as behavioral and demographic information used for Autism prediction. The proposed system incorporates multiple security measures to ensure data protection, system reliability, and user trust. These measures address various aspects including data security, access control, model integrity, and ethical considerations.

One of the primary security concerns is data privacy and confidentiality. The dataset used in this project may contain sensitive personal information such as age, gender, and medical-related attributes. To ensure privacy, the system follows data anonymization practices, where personally identifiable information is either removed or not stored. Additionally, data is handled in a controlled environment to prevent unauthorized access or leakage.

Another important aspect is secure data storage and handling. The dataset and trained model are stored locally in a secure environment. Access to these files is restricted to authorized users only. The model is saved using serialization (pickle), and care is taken to ensure that the file is not tampered with. Proper file permissions and restricted access help in maintaining data integrity and preventing unauthorized modifications.

The system also implements access control mechanisms. Only authorized users should be allowed to interact with the system, especially in a real-world deployment scenario. Authentication techniques such as username-password verification can be incorporated to restrict access. Role-based access control can also be implemented to differentiate between users such as administrators, researchers, and general users.

Input validation and error handling play a crucial role in system security. The

system ensures that all user inputs are validated before processing. Invalid or malicious inputs are detected and handled appropriately to prevent unexpected system behavior. This reduces the risk of injection attacks or data corruption. Proper error messages are displayed to guide users without exposing sensitive system information.

Another key measure is model security and integrity. The trained machine learning model is protected from unauthorized modifications. Since the model plays a crucial role in prediction, any tampering could lead to incorrect or biased results. The use of secure storage and version control ensures that only validated models are used in the system. Additionally, periodic checks can be performed to ensure model consistency and reliability.

The integration of Explainable AI (SHAP) also contributes indirectly to system security. By providing transparency in model predictions, it allows users to understand how decisions are made. This reduces the risk of hidden biases or unfair decision-making. Transparency builds trust and ensures that the system behaves ethically and responsibly.

The system also considers protection against data imbalance and bias. Techniques such as SMOTE are used to balance the dataset, ensuring that the model does not favor one class over another. This prevents biased predictions and improves fairness, which is an important aspect of secure and ethical AI systems.

In terms of deployment security, if the system is extended to a web-based application, additional measures such as HTTPS encryption, secure APIs, and firewall protection should be implemented. Encryption ensures that data transmitted between the user and the server is secure and cannot be intercepted by unauthorized parties.

Regular system monitoring and maintenance are also essential. Logs can be

maintained to track system activity and detect any unusual behavior. Periodic updates and testing help in identifying vulnerabilities and ensuring that the system remains secure over time.

Finally, ethical considerations and compliance are taken into account. The system is designed to assist in prediction and not to replace professional medical diagnosis. Proper disclaimers should be included to inform users about the limitations of the system. Ensuring ethical use of AI is an important aspect of overall system security.

Secure and Robust Programming:

Secure and robust programming is a fundamental aspect of developing reliable software systems, especially in applications involving sensitive data such as Autism prediction. It ensures that the system is protected against potential threats while maintaining stability, accuracy, and efficiency under different conditions.

One of the key principles of secure programming is input validation. In the proposed system, all input data whether from the dataset or user-provided values is validated before processing. This includes checking for missing values, incorrect data types, and out-of-range values. Proper validation prevents unexpected errors and protects the system from malicious or incorrect inputs that could disrupt model predictions or cause system failure.

Another important aspect is exception handling and error management. The system is designed to handle runtime errors gracefully without crashing. For example, errors during data loading, preprocessing, or model prediction are managed using appropriate exception handling techniques. This ensures that the system continues to function smoothly and provides meaningful feedback to the user instead of terminating abruptly.

The use of modular programming practices enhances both security and robustness. The system is divided into multiple modules such as data preprocessing, model training, evaluation, and explainability. Each module performs a specific function and operates independently. This separation of concerns reduces complexity, makes debugging easier, and prevents errors in one module from affecting the entire system.

Data integrity is also a crucial factor in robust programming. The system ensures that the dataset remains consistent and unaltered during processing. Techniques such as controlled preprocessing, proper data transformations, and validation checks help maintain data accuracy. This is important because any inconsistency in data can lead to incorrect predictions.

The system also follows safe model handling practices. The trained machine learning model is saved securely using serialization techniques such as pickle. Care is taken to load only trusted model files to prevent the execution of malicious code. Additionally, version control of models can be implemented to ensure that only validated and tested models are used in the system.

Another key aspect is performance optimization and efficiency. The system is designed to handle data processing and prediction efficiently, minimizing computational overhead. Efficient algorithms and optimized libraries such as NumPy, Pandas, and Scikit-learn are used to ensure fast and reliable execution. This contributes to the robustness of the system by preventing delays and performance bottlenecks.

Logging and monitoring further enhance system reliability. Logs can be maintained to track system activities, including data processing steps and prediction results. This helps in identifying errors, debugging issues, and ensuring transparency in system operations. Monitoring also allows early detection of anomalies or unexpected behavior.

The integration of Explainable AI (SHAP) contributes to robustness by providing insights into model decisions. By understanding how features influence predictions, developers can identify potential issues such as bias or incorrect feature importance. This improves the overall reliability and trustworthiness of the system.

Additionally, the system is designed with scalability and maintainability in mind. Clean coding practices, proper documentation, and structured implementation make it easier to update and extend the system in the future. New features or improvements can be added without disrupting existing functionality. Best practices such as restricting file access, avoiding hard-coded sensitive data, and ensuring safe handling of external libraries are followed. These measures protect the system from vulnerabilities and ensure safe operation.

7. COST ESTIMATION

Cost estimation is an essential part of project planning, as it helps in evaluating the financial feasibility and resource requirements of the system. For the project "Explainable AI-Based Autism Spectrum Disorder Prediction using Machine Learning with SHAP Analysis," the cost estimation primarily focuses on development, tools, human effort, and deployment considerations.

Since this project is developed as part of an academic internship using open-source technologies, the overall cost is relatively low. The development is carried out using Python and libraries such as NumPy, Pandas, Scikit-learn, Matplotlib, Seaborn, XGBoost, and SHAP, all of which are freely available. This significantly reduces software licensing costs.

The major component of cost estimation in this project is human resource effort. The development process includes data preprocessing, exploratory data analysis, model training, evaluation, and implementation of explainable AI techniques. Assuming a single developer working approximately 6–8 hours per day over a period of two months, the estimated effort can be calculated in terms of person-hours. If we consider an average rate (for estimation purposes), the development cost can be approximated accordingly.

Another component is hardware cost. The system is developed on a personal computer or laptop with moderate specifications (such as 8GB RAM, i5 processor, etc.). Since no high-end infrastructure or GPU-based computation is required, hardware costs remain minimal. In a real-world scenario, cloud-based infrastructure (such as AWS, Google Cloud, or Azure) may be used, which would add operational costs based on usage.

The project also involves data acquisition cost, which is negligible in this case as publicly available datasets are used. However, in real-world applications,

acquiring high-quality medical datasets may involve significant costs.

To estimate the project cost systematically, a simplified version of the COCOMO (Constructive Cost Model) can be considered. The COCOMO model estimates effort and cost based on the size and complexity of the software. In this case, the project falls under the organic mode, as it is relatively small, developed by a single developer, and does not involve highly complex or critical systems.

Using the basic COCOMO model:

Effort (Person-Months) = a × (KLOC)^b

Since this is a small-scale project, the estimated effort is low, resulting in a manageable cost. Although exact KLOC (thousands of lines of code) is not calculated, the project can be categorized as low to medium complexity.

Component	**Estimated Cost (INR)**
Software (Python, Libraries)	0 (Open Source)
Hardware (Laptop)	Existing Resource
Development Effort	₹15,000 – ₹25,000
Data Acquisition	0 (Public Dataset)
Deployment (Optional)	₹2,000 – ₹5,000
Maintenance (Optional)	₹1,000 – ₹3,000

Table 7.1: Estimated Cost Breakdown of the Proposed System

8. FUTURE SCOPE

The proposed system for Autism Disorder prediction using Machine Learning and Explainable AI provides a strong foundation for early screening and analysis. However, there are several opportunities for further enhancement and expansion to improve its functionality, scalability, and real-world applicability.

One of the primary areas of future scope is the development of a user-friendly web or mobile application. Currently, the system operates in a local development environment such as Jupyter Notebook. By deploying the model using frameworks like Flask, Django, or Streamlit, the system can be made accessible to a wider audience, including healthcare professionals, researchers, and general users. A well-designed interface would allow users to input data easily and receive predictions along with explanations in real time.

Another important enhancement is the integration of real-time data and larger datasets. The current system is trained on a publicly available dataset, which may have limitations in terms of size and diversity. In the future, incorporating larger and more diverse datasets collected from hospitals or research institutions can improve model accuracy and generalization. This would make the system more reliable for real-world applications.

The system can also be extended by incorporating advanced machine learning and deep learning techniques. Models such as Neural Networks, Gradient Boosting frameworks, or ensemble methods can be explored to further improve prediction performance. Additionally, techniques like feature engineering and automated machine learning (AutoML) can be used to optimize model selection and performance.

Another significant area of improvement is enhancing the Explainable AI (XAI) capabilities. While SHAP provides strong interpretability, additional techniques such as LIME (Local Interpretable Model-agnostic Explanations) can be

integrated to provide alternative perspectives on model predictions. Combining multiple XAI techniques can further increase transparency and user trust.

The system can also be integrated with healthcare systems and electronic medical records (EMR) to provide automated screening support for doctors and specialists. This would enable seamless data flow and assist professionals in making faster and more informed decisions. Integration with telemedicine platforms can further enhance accessibility, especially in remote areas.

From a security perspective, future enhancements can include advanced authentication and data encryption mechanisms to ensure safe handling of sensitive medical data. Implementing secure APIs and compliance with data protection standards would make the system more suitable for real-world deployment.

Another potential improvement is the addition of multi-language support and accessibility features. This would make the system more inclusive and usable by a diverse population, especially in regions with varying linguistic backgrounds.

The system can also be enhanced by adding predictive analytics dashboards and reporting tools. These dashboards can provide detailed insights, trends, and visual summaries of predictions, helping users better understand patterns in the data.

Finally, continuous monitoring and updating of the model is essential. Implementing a feedback mechanism where user input and outcomes can be used to retrain the model will improve its performance over time. This will ensure that the system remains accurate, up-to-date, and adaptable to new data.

9. APPENDICES

9.1 CODING

```
#!/usr/bin/env python

# coding: utf-8

# **1. Importing the dependencies**

# In[101]:

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

import shap

from sklearn.preprocessing import LabelEncoder

from imblearn.over_sampling import SMOTE

from sklearn.model_selection import train_test_split, cross_val_score, RandomizedSearchCV

from sklearn.tree import DecisionTreeClassifier

from sklearn.ensemble import RandomForestClassifier

from xgboost import XGBClassifier

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

import pickle
```

```
# **2. Data Loading & Understanding**

# In[102]:

# read the csv data to a pandas dataframe

df = pd.read_csv("train.csv")

# Initial Inspection

# In[103]:

df.shape

# In[104]:

df.head()

# In[105]:

df.tail()

# In[106]:

# display all columns of a dataframe

pd.set_option('display.max_columns', None)

# In[107]:

df.info()

# In[108]:

# convert age column datatype to integer

df["age"] = df["age"].astype(int)

# In[109]:

df.head(2)

# In[110]:

for col in df.columns:

  numerical_features = ["ID", "age", "result"]

  if col not in numerical_features:
```

```
    print(col, df[col].unique())

    print("-"*50)

# In[111]:

# dropping ID & age_desc column

df = df.drop(columns=["ID", "age_desc"])

# In[112]:

df.shape

# In[113]:

df.head(2)

# In[114]:

df.columns

# In[115]:

df["contry_of_res"].unique()
# In[116]:
# define the mapping dictionary for country names
mapping = {
    "Viet Nam": "Vietnam",
    "AmericanSamoa": "United States",
    "Hong Kong": "China"
}
# repalce value in the country column
df["contry_of_res"] = df["contry_of_res"].replace(mapping)
# In[117]:
df["contry_of_res"].unique()
# In[118]:
# taget class distribution
df["Class/ASD"].value_counts()
# **Insights:**
# 1. missing values in ethnicity & relation
```

```
# 2. age_desc column has only 1 unique value. so it is removed as it is not important for prediction
# 3. fixed country names
# 4. identified class imbalance in the target column
# **3. Exploratory Data Analysis (EDA)**
# In[119]:
df.shape
# In[120]:
df.columns
# In[121]:
df.head(2)
# In[122]:
df.describe()
# In[123]:
plt.figure()
df['age'].hist()
plt.title("Age Distribution")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()
# In[124]:
df['gender'].value_counts().plot(kind='bar')
plt.title("Gender Distribution")
plt.xlabel("Gender")
plt.ylabel("Count")
plt.show()
# In[125]:
df['Class/ASD'].value_counts().plot(kind='bar')
plt.title("Class Distribution (ASD vs Non-ASD)")
plt.xlabel("Class")
plt.ylabel("Count")
plt.show()
```

```
# In[126]:
import seaborn as sns
plt.figure()
sns.heatmap(df.corr(), annot=False)
plt.title("Correlation Heatmap")
plt.show()
# In[127]:
plt.figure()
df.boxplot()
plt.title("Boxplot for Outlier Detection")
plt.xticks(rotation=90)
plt.show()
# **Univariate Analysis**
# Numerical Columns:
# - age
# - result
# In[128]:
# set the desired theme
sns.set_theme(style="darkgrid")
# Distribution Plots
# In[129]:
# Histogram for "age"
sns.histplot(df["age"], kde=True)
plt.title("Distribution of Age")
# calculate mean and median
age_mean = df["age"].mean()
age_median = df["age"].median()
print("Mean:", age_mean)
print("Median:", age_median)
# add vertical lines for mean and median
plt.axvline(age_mean, color="red", linestyle="--", label="Mean")
plt.axvline(age_median, color="green", linestyle="-", label="Median")
plt.legend()
```

```
plt.show()
# In[130]
# Histogram for "result"
sns.histplot(df["result"], kde=True)
plt.title("Distribution of result")
# calculate mean and median
result_mean = df["result"].mean()
result_median = df["result"].median()
print("Mean:", result_mean)
print("Median:", result_median)
# add vertical lines for mean and median
plt.axvline(result_mean, color="red", linestyle="--", label="Mean")
plt.axvline(result_median, color="green", linestyle="-", label="Median")
plt.legend()
plt.show()
# **Box plots for identifying outliers in the numerical columns**
# In[131]:
# box plot
sns.boxplot(x=df["age"])
plt.title("Box Plot for Age")
plt.xlabel("Age")
plt.show()
# In[132]:
# box plot
sns.boxplot(x=df["result"])
plt.title("Box Plot for result")
plt.xlabel("result")
plt.show()
# In[133]:
# count the outliers using IQR method
Q1 = df["age"].quantile(0.25)
Q3 = df["age"].quantile(0.75)
IQR = Q3 - Q1
```

```
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
age_outliers = df[(df["age"] < lower_bound) | (df["age"] > upper_bound)]
# In[134]:
len(age_outliers)
# In[135]
# count the outliers using IQR method
Q1 = df["result"].quantile(0.25)
Q3 = df["result"].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
result_outliers = df[(df["result"] < lower_bound) | (df["result"] >
upper_bound)]
# In[136]:
len(result_outliers)
# Univariate analysis of Categorical columns
# In[137]:
df.columns
# In[138]:
categorical_columns = ['A1_Score', 'A2_Score', 'A3_Score', 'A4_Score',
'A5_Score', 'A6_Score',
    'A7_Score', 'A8_Score', 'A9_Score', 'A10_Score', 'gender',
    'ethnicity', 'jaundice', 'austim', 'contry_of_res', 'used_app_before',
    'relation']
for col in categorical_columns:
  sns.countplot(x=df[col])
  plt.title(f"Count Plot for {col}")
 plt.xlabel(col)
  plt.ylabel("Count")
  plt.show()
# In[139]:
```

```
# countplot for target column (Class/ASD)
sns.countplot(x=df["Class/ASD"])
plt.title("Count Plot for Class/ASD")
plt.xlabel("Class/ASD")
plt.ylabel("Count")
plt.show()
# In[ ]:
# In[140]:
df["Class/ASD"].value_counts()
# handle missing values in ethnicity and relation column
# In[141]:
df["ethnicity"] = df["ethnicity"].replace({"?": "Others", "others": "Others"})
# In[142]:
df["ethnicity"].unique()
# In[143]:
df["relation"].unique()
# In[41]:
df["relation"] = df["relation"].replace(
    {"?": "Others",
     "Relative": "Others",
     "Parent": "Others",
     "Health care professional": "Others"}
)
# In[42]:
df["relation"].unique()
# In[43]:
df.head()
# **Label Encoding**
# In[44]:
```

```
# identify columns with "object" data type
object_columns = df.select_dtypes(include=["object"]).columns
# In[45]:
print(object_columns)
# In[46]:
# initialize a dictionary to store the encoders
encoders = {}
# apply label encoding and store the encoders
for column in object_columns:
  label_encoder = LabelEncoder()
  df[column] = label_encoder.fit_transform(df[column])
  encoders[column] = label_encoder   # saving the encoder for this column
# save the encoders as a pickle file
with open("encoders.pkl", "wb") as f:
  pickle.dump(encoders, f)
# In[47]:
encoders
# In[48]:
df.head()
# Bivariate Analysis
# In[49]:
# correlation matrix
plt.figure(figsize=(15, 15))
sns.heatmap(df.corr(), annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation heatmap")
plt.show()
# **Insights from EDA:**
#
# - There are few outliers in the numerical columns (age, results)
# - There is a class imbalance in the target column
# - There is a class imbalance in the categorical features
```

```
# - We don't have any highly correlated column
# - performed label encoding and saved the encoders
# **4. Data preprocessing**
# Handling teh outliers
# In[50]:
# function to replace the outliers with median
def replace_outliers_with_median(df, column):
  Q1 = df[column].quantile(0.25)
  Q3 = df[column].quantile(0.75)
  IQR = Q3 - Q1
  lower_bound = Q1 - 1.5 * IQR
  upper_bound = Q3 + 1.5 * IQR
  median = df[column].median()
  # replace outliers with median value
  df[column] = df[column].apply(lambda x: median if x < lower_bound or x >
upper_bound else x)
return df
# In[51]:
# replace outliers in the "age" column
df = replace_outliers_with_median(df, "age")
# replace outliers in the "result" column
df = replace_outliers_with_median(df, "result")
# In[52]:
df.head()
# In[53]:
df.shape
# **Train Test Split**
# In[54]:
df.columns
# In[55]:

X = df.drop(columns=["Class/ASD"])
```

```
y = df["Class/ASD"]
# In[56]:
print(X)
# In[57]:
print(y)
# In[58]:
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
# In[59]:
print(y_train.shape)
print(y_test.shape)
# In[60]:
y_train.value_counts()
# In[61]:
y_test.value_counts()
# **SMOTE (Synthetic Minority Oversampling technique)**
# In[62]:
smote = SMOTE(random_state=42)
# In[63]:
X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)
# In[64]:
print(y_train_smote.shape)
# In[65]:
print(y_train_smote.value_counts())
# **5. Model Training**
# In[66]:
# dictionary of classifiers
models = {
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Random Forest": RandomForestClassifier(random_state=42),
    "XGBoost": XGBClassifier(random_state=42)
}
```

```
# In[89]:
# dictionary to store the cross validation results
cv_scores = {}
# perform 5-fold cross validation for each model
for model_name, model in models.items():
  print(f"Training {model_name} with default parameters...")
  scores = cross_val_score(model, X_train_smote, y_train_smote, cv=5,
scoring="accuracy")
  cv_scores[model_name] = scores
  print(f"{model_name} Cross-Validation Accuracy: {np.mean(scores):.2f}")
  print("-"*50)
# Train final model (choose best one)
from xgboost import XGBClassifier
model = XGBClassifier(random_state=42)
# Train on full training data
model.fit(X_train_smote, y_train_smote)
# In[68]:
cv_scores
# **6. Model Selection & Hyperparameter Tuning**
# In[69]:
# Initializing models
decision_tree = DecisionTreeClassifier(random_state=42)
random_forest = RandomForestClassifier(random_state=42)
xgboost_classifier = XGBClassifier(random_state=42)
# In[70]:
# Hyperparameter grids for RandomizedSearchCV
param_grid_dt = {
    "criterion": ["gini", "entropy"],
    "max_depth": [None, 10, 20, 30, 50, 70],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 2, 4]
}
```

```
param_grid_rf = {
    "n_estimators": [50, 100, 200, 500],
    "max_depth": [None, 10, 20, 30],
    "min_samples_split": [2, 5, 10],
 "min_samples_leaf": [1, 2, 4],
    "bootstrap": [True, False]
}
param_grid_xgb = {
    "n_estimators": [50, 100, 200, 500],
    "max_depth": [3, 5, 7, 10],
    "learning_rate": [0.01, 0.1, 0.2, 0.3],
    "subsample": [0.5, 0.7, 1.0],
    "colsample_bytree": [0.5, 0.7, 1.0]
}
# In[71]:
# hyperparameter tuning for 3 tree based models
# the below steps can be automated by using a for loop or by using a pipeline
# perform RandomizedSearchCV for each model
random_search_dt = RandomizedSearchCV(estimator=decision_tree, param_distributions=param_grid_dt, n_iter=20, cv=5, scoring="accuracy", random_state=42)
random_search_rf = RandomizedSearchCV(estimator=random_forest, param_distributions=param_grid_rf, n_iter=20, cv=5, scoring="accuracy", random_state=42)
random_search_xgb = RandomizedSearchCV(estimator=xgboost_classifier, param_distributions=param_grid_xgb, n_iter=20, cv=5, scoring="accuracy", random_state=42)
# In[72]:
# fit the models
random_search_dt.fit(X_train_smote, y_train_smote)
random_search_rf.fit(X_train_smote, y_train_smote)
random_search_xgb.fit(X_train_smote, y_train_smote)
```

```
# In[73]:
# Get the model with best score
best_model = None
best_score = 0
if random_search_dt.best_score_ > best_score:
  best_model = random_search_dt.best_estimator_
  best_score = random_search_dt.best_score_
if random_search_rf.best_score_ > best_score:
  best_model = random_search_rf.best_estimator_
  best_score = random_search_rf.best_score_
if random_search_xgb.best_score_ > best_score:
  best_model = random_search_xgb.best_estimator_
  best_score = random_search_xgb.best_score_
# In[74]:
print(f"Best Model: {best_model}")
print(f"Best Cross-Validation Accuracy: {best_score:.2f}")
# In[75]:
# save the best model
with open("best_model.pkl", "wb") as f:
  pickle.dump(best_model, f)
# **7. Evaluation**
# In[76]:
# evaluate on test data
y_test_pred = best_model.predict(X_test)
print("Accuracy score:\n", accuracy_score(y_test, y_test_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_test_pred))
print("Classification Report:\n", classification_report(y_test, y_test_pred))
# To do:
# 1. Build a Predictive system with encoders and model file
# 2. See if you could improve teh performance
# In[79]:
get_ipython().system('pip install shap')
```

```
# In[90]:
model.predict(X_test[:1])
# In[91]:
# adding XAI
# Create explainer
explainer = shap.TreeExplainer(model)
# Get SHAP values
shap_values = explainer.shap_values(X_test)
shap.summary_plot(shap_values, X_test)
shap.summary_plot(shap_values, X_test, plot_type="bar")
shap.initjs()
shap.force_plot(explainer.expected_value, shap_values[0], X_test.iloc[0])
```

9.2 BIBLIOGRAPHY

[1] Scikit-learn Documentation. Pedregosa, F., et al. "Scikit-learn: Machine Learning in Python." Journal of Machine Learning Research.

[2] Python Software Foundation. Python Official Documentation. https://docs.python.org

[3] Kaggle Dataset Repository. Autism Screening Data for Machine Learning Applications.

[4] Han, J., Kamber, M., & Pei, J. Data Mining: Concepts and Techniques. Morgan Kaufmann.

[5] Bishop, C. M. Pattern Recognition and Machine Learning. Springer.

[6] Géron, A. Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow. O'Reilly Media.

[7] IEEE Research Articles on Machine Learning in Healthcare Applications.

[8] NumPy, Pandas, Matplotlib, and Seaborn Official Documentation.

[9] Research articles on Autism Spectrum Disorder early screening methodologies.